

УДК 004.652
ББК 32.972.13
С50

Смирнов М.В. Проектирование хранилищ данных [Электронный ресурс]: Учебно-методическое пособие / Смирнов М.В. — М.: МИРЭА – Российский технологический университет, 2024. — 1 электрон. опт. диск (CD-ROM)

В учебно-методическом пособии рассмотрены основные принципы управления проектами хранилищ данных от момента постановки задачи, до оценки результатов проектирования. На основании предоставленной информации, студентам предлагается решить ряд практических задач-кейсов по тематике пособия, а также, ответить на вопросы, связанные с разными этапами организации проекта хранилища данных. В состав учебно-методического пособия входят семь тем, каждая из которых заканчивается рядом тестовых и практических заданий, выполнение которых способствует более глубокому пониманию материала. Пособие содержит 49 таблиц, моделей и схем, необходимых для теоретического и практического изучения предмета исследования.

Предназначено для обучающихся по направлению 09.04.02 «Информационные системы и технологии», а также других направлений, изучающих дисциплины: «Проектирование хранилищ данных», «Проектирование баз данных», «Управление данными».

Учебно-методическое пособие издается в авторской редакции.

Авторский коллектив: Смирнов Михаил Вячеславович

Рецензенты:

Комиссаров Юрий Валерьевич, к.т.н., ст. преподаватель, МОФ МФЮУ

Максимова Елена Александровна, д.т.н., профессор, доцент, МИРЭА – Российский технологический университет

Системные требования:

Наличие операционной системы Windows, поддерживаемой производителем.

Наличие свободного места в оперативной памяти не менее 128 Мб.

Наличие свободного места в памяти постоянного хранения (на жестком диске) не менее 30 Мб.

Наличие интерфейса ввода информации.

Дополнительные программные средства: программа для чтения pdf-файлов (Adobe Reader).

Подписано к использованию по решению Редакционно-издательского совета

МИРЭА — Российский технологический университет.

Объем: 3.83 мб

Тираж: 10

ISBN 978-5-7339-2258-4

© Смирнов М.В., 2024

© МИРЭА – Российский
технологический университет, 2024

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	5
1. ХРАНИЛИЩА ДАННЫХ. ОПРЕДЕЛЕНИЕ ХРАНИЛИЩ ДАННЫХ. ЭЛЕМЕНТЫ ХРАНИЛИЩА ДАННЫХ	6
1.1. Определение хранилища данных. Свойства хранилищ данных	6
1.2. Классическая трехуровневая архитектура хранилища данных.....	9
1.3. Модели хранилища данных Инмона и Кимбалла. Матрица критериев выбора модели	13
1.4. Вопросы для самостоятельного изучения по теме	17
1.5. Тестовые задания для самопроверки.....	17
2. АСПЕКТЫ РЕАЛИЗАЦИИ ПРОЕКТА ХРАНИЛИЩА ДАННЫХ.....	19
2.1. Проект разработки хранилища данных. Команда проекта разработки	19
2.2. Ключевые точки успеха проекта хранилища данных. Тупиковые сценарии проектов.....	23
2.3. Трехмерная модель хранения данных. Понятие Informational Package (информационный пакет)	27
2.4. Вопросы для самостоятельного изучения по теме	31
2.5. Тестовые задания для самопроверки.....	31
3. ШАБЛОНЫ ПРОЦЕССИНГОВОЙ АРХИТЕКТУРЫ ХРАНИЛИЩ ДАННЫХ. АРХИТЕКТУРА ХРАНИЛИЩА ДАННЫХ «ЗВЕЗДА». АРХИТЕКТУРЫ ХРАНИЛИЩ ДАННЫХ НА ОСНОВЕ HDFS	34
3.1. Паттерны «процессинговой» архитектуры хранилищ данных. Достоинства и недостатки	34
3.2. Архитектура традиционного хранилища данных «Звезда».....	38
3.3. Шаблоны архитектуры хранилищ данных на основе файловой системы Hadoop (HDFS) для «Больших данных»	44
3.4. Вопросы для самостоятельного изучения по теме	46
3.5. Тестовые задания для самопроверки.....	46
4. ПРОЦЕДУРЫ ETL (EXTRACT, TRANSFORM, LOAD).....	48
4.1. Определение ETL процессов, сферы применения, типовая последовательность действий	48
4.2. Пайплайны и требования к процессам ETL	51
4.3. Техники загрузки данных в хранилище, выявление ошибок при процессе валидации данных.....	55
4.4. Вопросы для самостоятельного изучения по теме	57

4.5. Тестовые задания для самопроверки.....	58
5. ТЕХНОЛОГИЧЕСКИЕ СТЕКИ ХРАНИЛИЩ ДАННЫХ ДЛЯ BIGDATA	60
5.1. Масштабируемое OLAP хранилища данных ClickHouse. Распределенное MPP хранилище данных PostgreSQL GreenPlum.....	60
5.2. Типовой технологический стек хранилища больших данных на основе фреймворка Apache Hadoop	63
5.3. Вертикальные хранилища данных Vertica.....	65
5.4. Вопросы для самостоятельного изучения по теме	66
5.5. Тестовые задания для самопроверки.....	67
6. ДОКУМЕНТНАЯ МОДЕЛЬ ХРАНИЛИЩА ДАННЫХ MONGODB	69
6.1. Архитектура и программный инструментарий хранилища данных MongoDB. Базовые модели хранилищ MongoDB.....	69
6.2. Архитектурные паттерны элементов хранилища MongoDB.....	73
6.3. Инструменты визуализации модели данных MongoDB	79
6.4. Вопросы для самостоятельного изучения по теме	82
6.5. Тестовые задания для самопроверки.....	82
7. ШАРДИРОВАННЫЕ КЛАСТЕРЫ ХРАНИЛИЩА ДАННЫХ MONGODB	84
7.1. Элементарная схема хранилища данных MongoDB. Проблема масштабирования хранилища для больших данных	84
7.2. Стандарт шардирования хранилища, варианты схем шардирования MongoDB.....	85
7.3. Вопросы для самостоятельного изучения по теме	89
ОТВЕТЫ НА ТЕСТОВЫЕ ЗАДАНИЯ	90
СПИСОК ЛИТЕРАТУРЫ.....	91
СВЕДЕНИЯ ОБ АВТОРЕ.....	92

ВВЕДЕНИЕ

Влияние математического, статистического анализа, а также машинного обучения, использующего эти методы, на быстрое принятие правильных тактических и стратегических решений в бизнесе трудно переоценить. При этом, массивы данных, которые необходимы для аналитики все чаще относятся к категории «большие данные», что накладывает дополнительные требования к моделям СУБД, которые создаются для их хранения. Внушительные потоки данных и огромные объемы их хранения создают серьезные вызовы для крупных корпораций и хозяйствующих структур. Для решения этих задач существует отдельный класс технологий хранения данных, - «хранилища данных», которые могут быть реализованы как в традиционной OLTP-OLAP парадигме, так и в новой поSQL модели хранения.

Правильно созданное и эффективно функционирующее хранилище данных является критически важным и быстрым источником требуемых для всех видов аналитической работы данных. При этом как проект разработки хранилища данных, так и последующая его эксплуатация – это длительная, сложная и комплексная задача, требующая консолидации усилий большого коллектива людей обладающими разными, подчас уникальными компетенциями.

В данном издании последовательно рассматриваются традиционные и современные подходы к реализации проектов хранилищ данных, от атомарных традиционных, до целых технологических стеков, построенных на современных поSQL технологиях. Отдельное внимание уделяется управлению участниками проекта, а также мониторингу ключевых маркеров успеха (и неудачи) проектов хранилищ данных.

Значительная часть материала посвящена вопросам подбора правильной архитектуры построения самого хранилища данных и его отдельных компонентов, поскольку правильно выбранная и построенная архитектура – это наиболее значительный критерий конечного успеха проекта.

Отдельно, в тексте рассмотрены наиболее распространенные технологические стеки для работы с большими данными, имеющие реальное практическое применение в крупнейших организациях мира.

1. ХРАНИЛИЩА ДАННЫХ. ОПРЕДЕЛЕНИЕ ХРАНИЛИЩ ДАННЫХ. ЭЛЕМЕНТЫ ХРАНИЛИЩА ДАННЫХ

1.1. Определение хранилища данных. Свойства хранилищ данных

К настоящему времени, в научных литературных источниках сформировалось достаточно большое количество разных трактовок понятия хранилище данных. Это в первую очередь связано с быстрой эволюцией технологий хранения данных (в том числе и с переходом человечества в парадигму BigData) и вносит определенные сложности в определении актуальной трактовки. Тут и далее, автор пособия будет опираться на два классических определения хранилищ данных, введенными в оборот учеными – создателями теории и первых практических решений в области хранилищ, Б. Инмона и Ш. Келли.

Билл Инмон определяет *хранилища данных*, как предметно-ориентированные, интегрированные, независимые и управляемые временем коллекции данных, используемые при поддержке принятия решений.

В свою очередь, **Шон Келли** определяет хранилища данных по данным, хранимым в них. Данные в *хранилищах данных*: независимы, делимы, доступны, интегрированы, имеют временные метки, объектно-интегрированы.

В любом случае, какое из приведенных определений не брать за основу, становится очевидно, что конструкция хранилищ данных, с точки зрения технологий хранения данных, является уникальной, самостоятельной и не зависящей ни от одной из общепринятых сегодня парадигм хранения данных. Так, в отличие от реляционной парадигмы, хранилища данных в своих свойствах не ограничиваются требованиями о целостности и согласованности данных, строгости и неделимости схем реляционных таблиц, включая обязательство нормализации таблиц для исключения аномалий. При этом, в отличие от постреляционной парадигмы, хранилища данных подразумевают разумную интеграцию и упорядоченность данных, доступных в хранилище. Таким образом, хоть хранилище и строится с использованием инструментария программного обеспечения реляционной или постреляционной парадигмы хранения данных (что будет наглядно продемонстрировано далее в тексте последующих лекций и практических работ), называть хранилища вариацией реляционной или постреляционной модели хранения данных неправильно.

Далее, определим основные *свойства* хранилищ данных.

1. **Предметно-ориентированные данные.** Хранилища данных имеют

принципиально иные источники данных, нежели традиционные модели хранения. Основным источником данных для традиционных баз данных являются приложения (desktopные, мобильные, веб и т.д.). Пользователи, в рамках работы с приложениями эти данные вводят (генерируют) и сохраняют, как результат своей работы в документах или таблицах баз данных. Поскольку основная цель функционирования хранилищ данных не обслуживание пользовательских приложений, а обслуживание аналитических систем поддержки принятия решений (см. определение Инмона), то источниками данных для них уже являются не приложения, а реальные бизнес-объекты (цеха, отделы, филиалы или же предприятия в целом) или же бизнес-события (малые, средние и крупные бизнес-процессы).

2. **Интегрированные данные.** Мы привыкли к тому, что модель хранения данных в традиционной реляционной парадигме определяют объекты хранения (таблицы) и связи между ними. Правило простое – один объект наблюдения формирует один объект хранения. Достичь этого несложно, если набор входных данных поступает из одного источника, полностью согласован и направлен на решение прикладных задач какой-то предметной области. В хранилищах данных, в свою очередь, массив данных представляет собой комплексный агрегат из разнородных, синхронизированных друг с другом данных, получаемых из разных источников (порой, принципиально отличающихся друг от друга). Рассмотрим пример элемента хранилища данных «Банковский счет клиента». По своей сути, — это сложный агрегат, собираемый как минимум из трех потоков данных: информация о банковских вкладах клиента, информация о займах клиента и информация о балансе счета клиента. Очевидно, что эти потоки генерируются разными бизнес-процессами банка и разными его сервисами. В рамках процедур, связанных с функционированием хранилища данных, каждый из перечисленных в примере потоков данных синхронизируется с остальными, входящими в этот агрегат. Под синхронизацией, в первую очередь понимается приведение к единому виду массивов данных – их названия, используемые в них коды и системы кодирования, типы данных и единицы измерения.

3. **Изменяющиеся во времени данные.** Мы все помним одно из главных требований реляционной модели данных – это согласованность данных. Оно означает, что все данные, находящиеся в базе, должны быть согласованы в одном моменте времени, «здесь и сейчас». Со временем, в рамках инструментариев реляционной и постреляционной модели данных появились паттерны, позволяющие хранить и обслуживать архивные данные (которые

априори не являются согласованными), но их роль до сих пор утилитарна. Учитывая же направленность хранилищ данных на аналитические и прогнозные исследования, как конечный результат функционирования (например, паттерны поведения покупателя: текущие и прогнозные), данные, находящиеся относительно друг друга в разных временных срезах (исторические, текущие, планируемые) – это основной массив данных, который будет составлять хранилище.

4. **Независимые данные.** Специфика наполнения и функционирования хранилищ данных подразумевает, что оно будет наполняться данными порционно, в определенные интервалы времени. Это снижает нагрузку от постоянных операций получения-загрузки данных и оставляет операционные мощности сервера хранилища данных большую часть времени свободными. В зависимости от требований бизнес-процессов, итерации загрузки данных могут производиться один раз в день, один раз в неделю, один раз в две недели. В некоторых случаях этот интервал может быть еще больше. Причем, для разных наборов данных в хранилище могут быть настроены разные интервалы данных. Например, описания продуктов для каталога можно обновлять раз в две недели, а то и реже, тогда как данные о продажах этих продуктов обновляются значительно чаще (например, каждый день). Стоит отметить, что с развитием больших данных и методов их обработки (потокковая передача больших данных или *стриминг*), в хранилищах данных (особенно организованных в формате облачного хранилища) все чаще практикуется передача данных в режиме «реального времени».

5. **Гранулярность данных.** В традиционных, реляционных моделях хранения практикует подход, когда данные собираются и хранятся с максимально доступным уровнем детализации (чтобы ничего не потеря). Например, если мы собираем данные для базы данных отдела продаж компании, мы будем стараться «вытянуть» в реляционные таблицы данные с конкретной кассы магазина, с конкретного магазина нашей сети и т.д... Контекст аналитических и прогнозных исследований подразумевает более абстрактный подход к данным, например – предварительную их агрегацию. Это означает, что один и тот же массив данных может быть представлен в хранилище с разной степенью абстракции (от менее укрупненных, более детализированных данных, до более укрупненных – предварительно агрегированных данных). Ниже, на рис. 1 показан пример разной степени гранулярности данных для данных о транзакциях пользователя в банковском хранилище данных.

<i>Дневная детализация</i>	<i>Месячная детализация</i>	<i>Квартальная детализация</i>
- Аккаунт	- Аккаунт	- Аккаунт
- Даты транзакций	- Месяц	- Месяц
- Количество	- Количество транзакций	- Количество транзакций
- Внесение/снятие средств	- Снятия средств	- Снятия средств
	- Внесения средств	- Внесения средств
	- Баланс на начало	- Баланс на начало
	- Баланс на конец	- Баланс на конец

Рисунок 1. Три степени грануляции данных в банковском хранилище данных

Специалист по базам данных обязательно должен учитывать, что перечисленные выше свойства существенно отличают подходы по проектированию и эксплуатации хранилищ данных от привычных методов, применяемых при работе в рамках реляционной парадигмы. Для успешной работы с хранилищами данных обязательно необходимо выработать дополнительные soft и hard-skills даже в привычном программном обеспечении баз данных.

1.2. Классическая трехуровневая архитектура хранилища данных

Современная наука, в рамках типовой модели архитектуры хранилища данных (от которой отталкиваются при разработке конкретного проекта) определяет так называемую трехуровневую (three-tier) модель архитектуры. Называется она так потому, что сама модель состоит из трех составных частей, или уровней (tier): нижний уровень (bottom tier), средний уровень (middle tier) и верхний уровень (top tier). Такое деление на три части унаследовано из широко распространенных архитектур построения программных средств (например, веб-сервер – сервер приложений – клиентский уровень), что является более близким и понятным для специалистов, пришедших в проектирование хранилищ данных из смежных ИТ-отраслей.

В общем виде, трехуровневая архитектура хранилища данных показана на рис. 2.



Рисунок 2. Трехуровневая архитектура хранилища данных в общем виде

Нижний уровень архитектуры включает в себя источники информации, которые будут заполнять хранилище данных. Это могут быть промежуточные (операционные) результаты работы компании, сохраненные в реляционном или ином виде в базах данных, это могут быть внешние потоки структурированных и неструктурированных данных, «плоские» файлы и любые другие источники данных. Проходя настроенные специалистами процедуры очистки и стандартизации данных (извлечение/очистка/изменение/загрузка/обновление или extract/clean/transform/load/refresh), данные направляются в непосредственно хранилище данных, которое находится на этом же уровне модели. Помимо самого хранилища, отдельного репозитория метаданных (поскольку метаинформации у хранилища данных на порядок больше, чем у любой реляционной базы данных сравнимого масштаба, требуется управляемый отдельный репозиторий) на этом уровне присутствует инструментарий администрирования и мониторинга, а также, если это необходимо – настроенные витрины данных.

Витрина данных – это фрагмент массива данных из хранилища, выделенный из него на временной или постоянной основе для проведения

аналитики данных по заранее определенным темам (например, витрина для аналитики по продажам, витрина для аналитики по складам и т.д.).

Более подробно, основные компоненты нижнего уровня архитектуры показаны на схеме, на рис. 3.

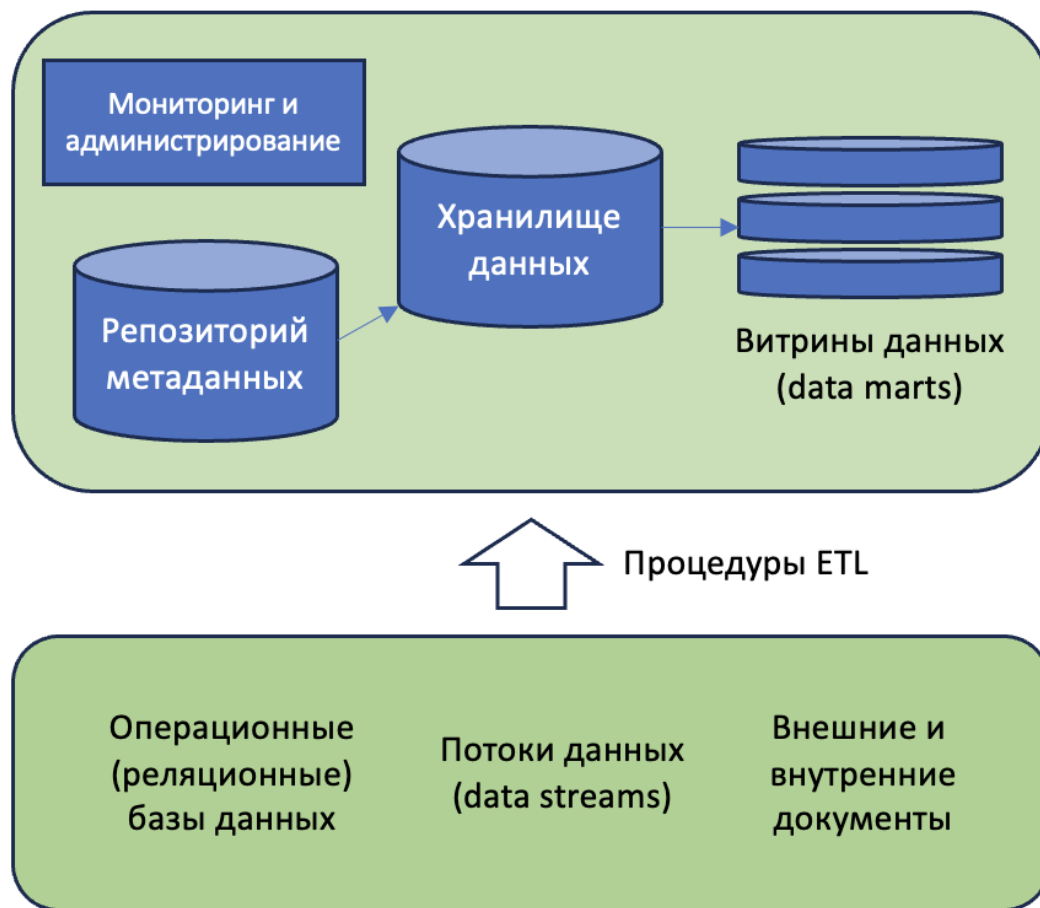


Рисунок 3. Нижний уровень (bottom tier) трехуровневой архитектуры хранилища данных

Результатом функционирования нижнего уровня архитектуры являются упомянутые выше витрины данных, или же выгрузки данных, передаваемые на следующий уровень архитектуры, на серверы OLAP (будет рассмотрено ниже, при определении среднего уровня архитектуры). Формирование витрин данных происходит там же, на нижнем уровне архитектуры. Схема движения информации при использовании в хранилище витрин данных показана на рис. 4. Предполагается, что технологией обработки данных выбрана стандартная OLTP и в процессе движения информации от источника к конечному потребителю она не меняется.

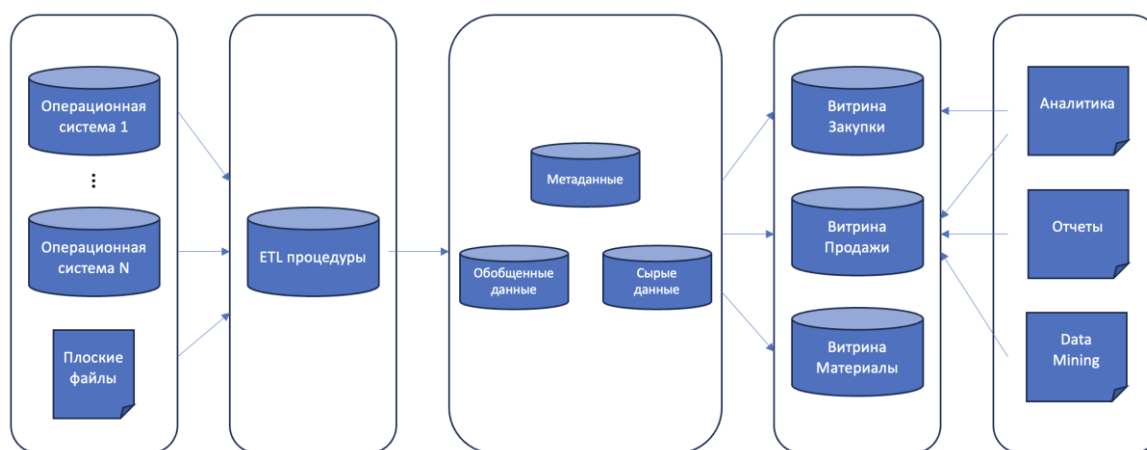


Рисунок 4. Движение информации при использовании витрин данных

Как было указано выше, витрины данных могут формироваться как временно, так и перманентно, в последнем случае работая по аналогии с серверами-репликами баз данных. Поскольку технология обработки остается неизменной (родственная реляционной модели хранения OLTP), конечные потребители, в лице аналитиков (графическая и табличная отчетность, аналитические выкладки, добыча данных) могут напрямую обращаться к сформированным витринам из приложения или из среды управления базами данных (с помощью SQL-запросов) и получать результаты напрямую из нижнего уровня архитектуры хранилища. Приведем условия применения витрин данных вместо одного или серии прямых SQL-запросов напрямую в базе данных.

1. Необходимость изоляции данных. Актуально, когда процесс обновления данных в базе занимает значительное время (например, из-за объема обновления) и может занимать от нескольких часов. В случае применения транзакционных запросов в обычной реляционной модели, традиционные блокировки могут создать значительное количество проблем работе аналитиков. Возможность изолировать данные в рамках созданной витрины данных позволяют свободно с ними работать даже в процессе длительной загрузки или обновления данных в самом хранилище с исключением «грязного чтения» обновляемых данных. Этот процесс во многом схож с применением репликации.

2. Гарантии атомарности. В случае сбоев и ошибок при загрузке или обновлении данных, витрина данных всегда будет оставаться в состоянии, которое предшествовало сбою в системе. Гарантии аналогичные реляционной модели хранения и транзакциям – или данные обновляются правильно и полностью, или не обновляются вовсе.

3. Системная темпоральность. Одно из ключевых отличий хранилищ

данных от традиционных реляционных баз – это возможность ведения системного времени и версионирование записей внутри хранилища. Витрина данных может сохранять свое состояние в разные моменты времени, что предоставляет исследователю возможность сравнивать данные, которые были в витрине в разные моменты времени, что существенно расширяет исследовательский потенциал аналитика. Данная функция, впрочем, может быть полезна и для администратора хранилища данных.

4. Витрины могут быть созданы как на нижнем уровне архитектуры хранилища для множества одновременных простых SQL-запросов (OLTP-нагрузка) или же могут быть созданы на OLAP-серверах среднего уровня архитектуры хранилища, что предоставит возможность исследователям использовать тяжелые аналитические «drilling» запросы (OLAP-нагрузка). Про разные виды запросов и нагрузок будет подробно рассказано ниже, а также на практических занятиях курса.

Средний уровень архитектуры. Если необходимо, на нем располагаются OLAP-серверы. Это программное обеспечение, аппаратное обеспечение или их сочетание, задачей которых является представление данных в многомерном виде. Грубо говоря, OLAP-сервер превращает традиционную модель хранения с таблицами и связями в модель хранения с измерениями и фактами (структура многомерных хранилищ данных будет подробно рассмотрена далее в соответствующей теме). Данные серверы могут работать в реляционной, многомерной или смешанной парадигме. Далее по тексту преимущество будет отдаваться реляционной модели OLAP-сервера (ROLAP).

Верхний уровень архитектуры – это программное обеспечение, используемое аналитиками для обработки и конечного анализа извлеченных данных. Данное ПО широко представлено различными вендорами, а выбор программных продуктов, работающих в OLAP и OLTP-нагрузке в основном зависит от поставленной перед аналитиками задачи.

1.3. Модели хранилища данных Инмона и Кимбалла. Матрица критериев выбора модели

В настоящий момент времени, среди проектов хранилищ данных, построенных на основании реляционной модели хранения (про постреляционную парадигму поговорим далее отдельно) выделяются две основных модели, которые были названы по фамилиям их создателей: модель

Инмона и модель Кимбалла. Далее приведем основные компоненты и особенности этих вариантов построения хранилищ данных.

Модель хранилища данных Инмона в большей степени опирается на базовые постулаты теории реляционных баз данных, что подразумевает максимальное использование свойств реляционных баз данных ACID (в первую очередь - атомарности), а также процедуры, связанные с нормализацией таблиц хранилища при его моделировании. Абстрактная схема хранилища Инмона показана на рис. 5.

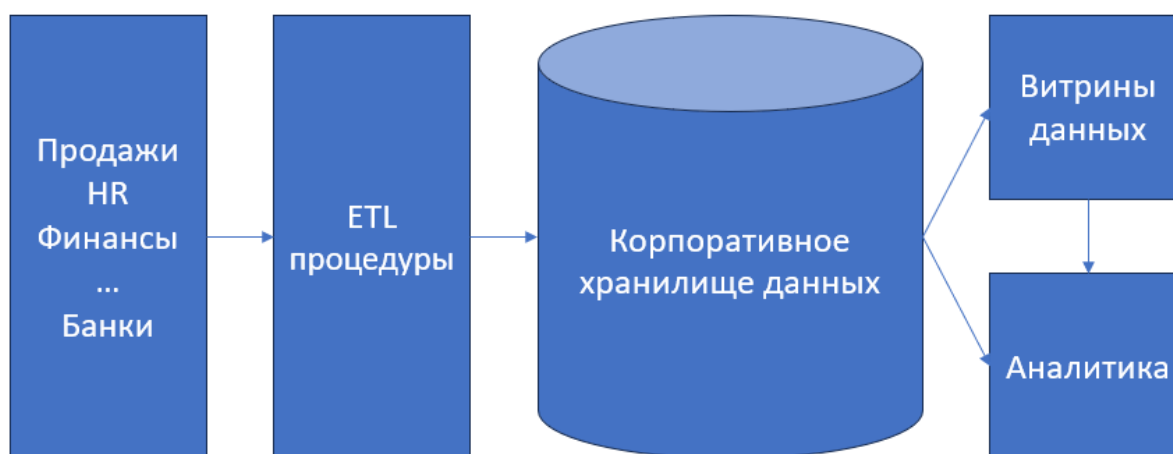


Рисунок 5. Структура модели хранилища данных Инмона

Корпоративные приложения или системы источника – блок в левой части рисунка, это системы, которые используются в операционных процессах организаций и создают данные, которыми впоследствии наполняется хранилище. Данные системы, как правило, отражают активность разных отделов хозяйствующих субъектов (продажи, управление персоналом, маркетинг, производство и пр.).

ETL процедуры или, в данной модели, - сервисы данных (data services). В рамках установленного формата, правил или политик предприятий, на этом этапе многообразие данных из корпоративных приложений конвертируется в очищенный от ошибок и аномалий массив данных строго регламентированного образца. Процессы извлечения, обработки и загрузки данных в хранилище в модели Инмона могут выполняться в режиме пакетной передачи (batch), в режиме реального времени, или в гибридном режиме, в зависимости от особенностей функционирования и предметной области организации.

Корпоративное хранилище данных. Центральный элемент архитектуры, который содержит атомарные данные, находящиеся в третьей нормальной форме.

По структуре хранения данных, это хранилище не отличается от стандартной базы данных, построенной в рамках реляционной модели.

Витрины данных. Заранее выбранные с помощью инструментов агрегации «тематические» массивы данных. Как правило, представлены в виде постоянно функционирующих реплик центрального хранилища. Отдельными витринами могут быть: витрина маркетинговых данных, витрина финансовых операций, витрина продаж и т.д... Аналитические приложения, находящиеся в правой части схемы, могут обращаться за массивами данных как к центральному хранилищу напрямую, так и к соответствующим цели исследования витринам данных.

Модель хранилища данных Кимбалла, в свою очередь, полностью отражает все особенности OLAP процессинга данных. Она значительно меньше зависит от свойств реляционной модели хранения, а некоторые из них в этой модели абсолютно неприменимы. Ключевым элементом модели является хранилище данных, построенное по принципу измерений. Абстрактная схема хранилища Кимбалла показана на рис. 6.

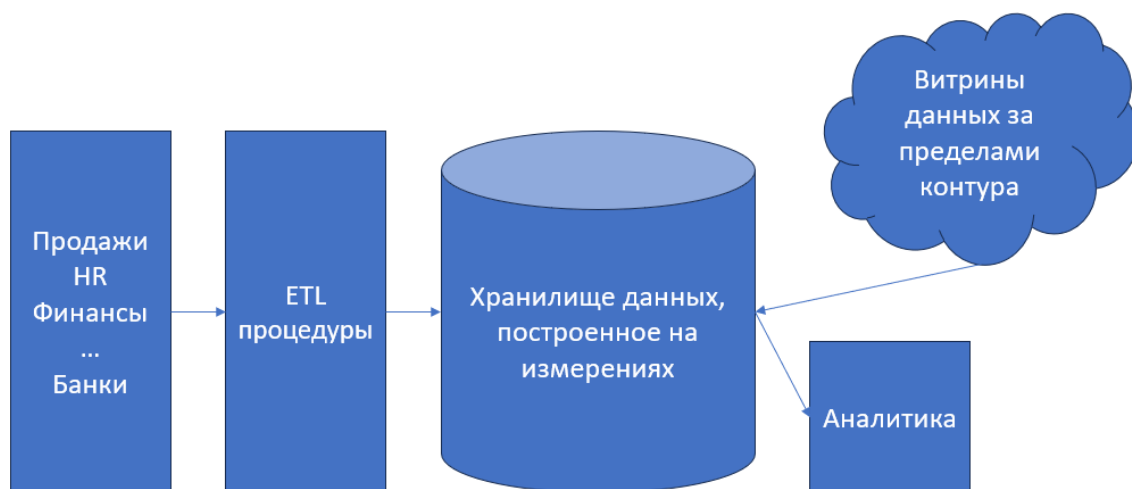


Рисунок 6. Структура модели хранилища данных Кимбалла

Транзакционные приложения, аналогично модели Инмона находятся в левой части контура модели и собирают информацию в рамках операционного функционирования организации.

ETL-процедуры, также аналогично модели Инмона, собирают данные из разных транзакционных приложений, очищают от аномалий и ошибок, стандартизируют в рамках принятой схемы и, наконец, передают в центральное хранилище.

Хранилище данных, построенное на измерениях. Содержит корпоративные данные в виде, наиболее удобном для проведения процедур «data

drilling». Это один из методов анализа данных, когда исследователь углубляется в исследование массива данных, конкретизируя запрос на каждой последующей итерации (переходит на другой уровень грануляции). Для обеспечения этих процедур, при построении хранилища используется модель, когда все таблицы становятся измерениями (подробнее об этом в последующих лекциях). Модель может быть организована по типу «звезда» или «снежинка». Как правило, аналитические системы, при применении этой модели, извлекают данных напрямую из хранилища, а не из витрин данных, как в случае с моделью Инмона.

Витрина данных (datamart). В данной модели чаще всего выполняет утилитарные функции, существует за пределами хранилища (в репликационном виде).

Выбор одной из двух приведенных моделей осуществляется при реализации проекта разработки хранилища данных архитектором проекта, и зависит от многих факторов, большинство из которых приведены на рис. 7.

Характеристика	Kmb	Inm
Уровень бизнес-решений	Тактический	Стратегический
Требования по интеграции данных	Отдельные бизнес-требования	Корпоративные бизнес-требования
Структура данных	Простые количественные данные, KPI	Многослойные данные, качественные данные
Стабильность данных	Источники редко меняются	Источники часто меняются
Сложность исполнения	Маленькая команда проекта	Большая команда проекта
Время на развертывание	Срочно требуется хранилище	Есть существенное время на проектирование и релиз
Затраты	Сравнительно низкие затраты	Высокие затраты со старта

Рисунок 7. Критерии выбора модели хранилища данных

Важно отметить, что, ввиду существенной разницы двух моделей, после реализации проекта хранилища данных, процесс замены одной модели на другую сравним по затратам с целым проектом разработки, поэтому к обоснованию выбора модели следует подходить максимально ответственно.

1.4. Вопросы для самостоятельного изучения по теме

(кейс) Вы – аналитик проектной команды, задачей которой является создание хранилища данных для страховой компании. Перечислите все возможные источники данных, из которых вы можете брать данные для вашего хранилища. Рассмотрите все возможные типы источников (корпоративные приложения, рис. 5-6). Дайте пояснение каждому выбранному источнику и потоку данных. Отчет оформить в шаблоне документа (форма документа доступна в группе предмета и будет предложена преподавателем отдельно).

1.5. Тестовые задания для самопроверки

1. Термин «интегрированные данные», применительно к хранилищам данных характеризует:

- А) условия работы с большим количеством разнородных источников данных на входе хранилища;
- Б) условия работы с агрегацией данных на входе хранилища;
- В) необходимость интеграции данных в реляционную схему хранилища.

2. Нижний уровень трехуровневой архитектуры хранилища данных отвечает за:

- А) представление данных в виде гиперкубов;
- Б) сбор, ETL процессы и хранение «сырых» данных;
- В) формирование массивов данных для конечных пользователей.

3. Средний уровень трехуровневой архитектуры хранилища данных отвечает за:

- А) представление данных в виде гиперкубов;
- Б) сбор, ETL процессы и хранение «сырых» данных;
- В) формирование массивов данных для конечных пользователей.

4. Как называется центральное хранилище данных в схеме «по Кимбаллу»?

- А) корпоративное хранилище данных;
- Б) аналитическое хранилище данных;
- В) хранилище данных, построенное на фактах;
- Г) хранилище данных, построенное на измерениях.

5. Как называется центральное хранилище данных в схеме «по Инмону»?

- А) корпоративное хранилище данных;
- Б) аналитическое хранилище данных;
- В) хранилище данных, построенное на фактах;
- Г) хранилище данных, построенное на измерениях.

6. Как называется процесс загрузки данных для схем noSQL хранилищ данных?

- А) ETL;
- Б) ELT;
- В) Bulk Load;
- Г) ни один из перечисленных вариантов.

7. Как называется процесс загрузки данных для традиционных схем хранилищ данных?

- А) ETL;
- Б) ELT;
- В) Bulk Load;
- Г) ни один из перечисленных вариантов.

8. Что означает параметр проекта хранилища данных «стабильность данных»?

- А) высокий или низкий уровень доверия к данным источника;
- Б) редкие или частые обновления хранилища данных;
- В) источники входных данных редко или часто меняются;
- Г) данные согласованы или не согласованы.

9. Какому типу хранилищ данных соответствует тактический (близкий горизонт принятия решений) уровень бизнес-решений?

- А) хранилище Кимбалла;
- Б) хранилище Инмона;
- В) аналитическое хранилище;
- Г) хранилища данных не используются для аналитики тактического уровня, для этого достаточно реляционных БД.

2. АСПЕКТЫ РЕАЛИЗАЦИИ ПРОЕКТА ХРАНИЛИЩА ДАННЫХ

2.1. Проект разработки хранилища данных. Команда проекта разработки

Проект разработки хранилища данных, с точки зрения методологии, мало чем отличается от проектов разработки программного обеспечения информационных систем и информационных технологий. В зависимости от поставленных в техническом задании, проект может реализовываться как в классическом, последовательном виде, так и в «гибких» сценариях разработки. Этапы жизненного цикла хранилища данных также, в целом, схожи с жизненным циклом программного продукта и показаны на рис. 8.



Рисунок 8. Типовой жизненный цикл проекта хранилища данных

Выделяют пять основных стадий жизненного цикла, за реализацию которых отвечает **команда проекта**: планирование, постановка задачи, проектирование и построение хранилища, развертывание и эксплуатация, вплоть до вывода хранилища данных из эксплуатации.

На этапе **планирования проекта** определяется предметная область проектирования, изучению подлежит объект исследования (организация, для которой будет разрабатываться хранилище данных). Исследование целесообразно проводить инженерам, входящим в группу разработки, ведь его результатом, как правило, является набор формальных требований к проекту.

На этапе **определения требований** к проекту происходит конвертация неформальных требований, собранных в ходе изучения объекта исследования в формальные требования, которые удобно использовать как КРІ при оценке результатов проекта, так и как исчерпывающее, однозначное, максимально

подробное руководство для проектировщиков.Arteфактом этого этапа является техническое задание на проектирование хранилища данных.

В ходе *проектирования и построения хранилища*, команда разработчиков сперва создает и максимально оптимизирует модель будущего хранилища данных, после чего последовательно превращает ее сперва в скрипт языка программирования, а затем – готовит к развертыванию на тестовых мощностях заказчика проекта. В ходе реализации этого большого и значимого этапа, как и в случае с программным обеспечением, создается сопроводительная документация, вспомогательные эскизы интерфейсов и листинги программного кода. Также осуществляется модульное тестирование компонентов хранилища (хранимых процедур, процедур ETL), если это необходимо.

Готовое хранилище разворачивается на мощностях заказчика в ходе релиза или *развертывания*. После подготовки соответствующего помещения, сборки и настройки «железа», установки необходимого программного обеспечения, включая операционные системы на серверы, происходит выполнение листингов программного кода и создание рабочей физической модели хранилища данных. Оно заполняется операционными или тестовыми данными и проходит пуско-наладочные испытания.

Стадия *эксплуатации*, в первую очередь, связана с гарантийным и послегарантийным обслуживанием компонентов хранилища данных. Также, в ходе этого самого длительного этапа жизненного цикла хранилища данных оно проходит постоянную модернизацию, логично завершая этот цикл архивированием и переносом аккумулированных данных и выводом действующей модели хранилища из эксплуатации.

Одной из наиболее важных задач, которые решаются в ходе планирования проекта хранилища данных является определение участников проекта и их компетенций, которые способны решить все поставленные перед группой проекта задачи. Данная задача осложняется тем, что основная масса профессиональных проектировщиков данных в ходе своей профессиональной деятельности имеет дело с традиционной OLTP моделью хранения данных (реляционные базы данных) и когда компания принимает решение о переходе к OLAP модели хранилища данных обязательно встает вопрос профессиональной квалификации имеющихся ИТ-специалистов. Необходимо или переучивать имеющихся, или искать на рынке труда сформировавшихся профессионалов.

Далее, определим возможные роли проекта хранилища данных и кратко

опишем набор трудовых задачи каждой из ролей.

Исполнительный спонсор (директор проекта, executive sponsor), лицо, принимающее решения в ходе реализации проекта. К числу его ключевых обязанностей относится поддержка и отстаивание интересов проектной команды у заказчика, финансирующего проект. Разрешение спорных ситуаций, возникающих в ходе реализации проекта, - также в компетенции этого специалиста. Модельные навыки специалиста: хорошее знание предметной области проекта, энтузиазм и высокая психологическая устойчивость.

Проектный менеджер (project manager) – ответственный за ход реализации проекта, на которого возложены обязанности мониторинга хода его выполнения. В его компетенции находится формирование распоряжений и контроль их выполнения. Модельные навыки специалиста: навыки тайм-менеджмента, знание актуальных методологий разработки программного обеспечения, глубокий опыт управления проектами.

Менеджер по связям с пользователями (user liaison manager) – всесторонняя координация с конечными пользователями хранилища данных в ходе всего проекта разработки с возможностью внесения корректировок в проект. Модельные навыки: уважение в среде конечных пользователей программного продукта, высокий уровень организационных навыков, понимание точки зрения конечного пользователя продукта.

Ведущий архитектор хранилища (lead architect) – рассмотрение разных вариантов реализации хранилища данных, учитывая требования конкретного проекта разработки и выбор оптимальной архитектуры. Его главной задачей является составление максимально подробного и понятного архитектурного проекта хранилища данных (модель хранилища на верхнем уровне абстракции) в виде документа. Модельные навыки: навык масштабирования моделей проекта от верхнего уровня абстракции до максимально подробной декомпозиции, хорошо развитые аналитические навыки, «евангелизм» (высокий уровень знания современных информационных технологий и устоявшееся собственное мнение по их применению).

Специалист по инфраструктуре хранилища (infrastructure specialist) – проектирование всей необходимой для развертывания хранилища данных инфраструктуры (в которую входит «серверное и клиентское железо», операционные системы и сопутствующее программное обеспечение, сетевое оборудование и пр.). Этот же специалист руководит процессом релиза артефакта хранилища данных на мощностях заказчика проекта. Модельные навыки: хорошее знание актуального «железа» на рынке, высокий уровень знаний о

принципах работы современных операционных систем и системных приложений, обширный опыт работы с серверным оборудованием и программным обеспечением в качестве пользователя.

Бизнес-аналитик (business analyst). Основной задачей является изучение бизнес-процессов на стороне заказчика и интерпретация их в более формальную модель для проекта. Им должны быть сформулированы четкие бизнес-правила и бизнес-ограничения, впоследствии закладываемые в логику работы хранилища данных, утилит ETL, итоговых витрин данных. Модельные навыки: большой опыт взаимодействия с конечными пользователями в разных предметных областях, большой опыт бизнес-аналитики и глубокое знание основных нотаций описания бизнес-процессов и бизнес-логики организаций.

Инженер данных (data modeler) – специалист по логическим моделям хранилища данных. Его основная задача – проектирование таблиц фактов и наборов измерений для решения поставленных в техническом задании задач. Модельные навыки: опыт аналитика данных, большой опыт работы как с OLAP, так и с OLTP моделями баз и хранилищ данных. Навыки построения распределенных (кластеры) баз данных.

Администратор хранилища данных (data warehouse administrator) – основная роль в ходе эксплуатации хранилища данных. В роли советника участвует в ходе реализации проекта хранилища данных. Обеспечивает бесперебойную работу хранилища данных в ходе эксплуатации, организывает мероприятия, связанные с обеспечением безопасности данных. Модельные навыки: опыт администрирования хранилищ и баз данных, глубокие знания элементов физической модели баз данных и хранилищ данных.

Специалист по трансформации данных (data transformation specialist) – ответственный за сборку и настройку ETL (extract, transform, load) процедур для хранилища данных и всех их производных. Модельные навыки: глубокое понимание типовых структур данных и потоков данных, понимание принципов работы разных источников информации, большой опыт практических навыков работы в программном обеспечении, связанным с ETL процессами.

Специалист по качеству данных (quality assurance analyst) – разработка и имплементация процедур, связанных с очисткой и верификацией данных, которые находятся непосредственно в хранилище данных. Модельные навыки: опыт мероприятий, связанных с повышением качества данных, глубокое понимание принципов работы источников информации и потоков данных.

Координатор по тестированию (testing coordinator) - разработка и проведение мероприятий по тестированию в ходе разработки и в ходе релиза

хранилища данных. Тестированию, помимо самого хранилища подвергаются программы, системы и сопутствующие инструменты хранилища данных. Модельные навыки: знание актуальных стандартов и методов тестирования, опыт работы с программным обеспечением тестирования, понимание принципов работы «входов и выходов» хранилища данных, навыки программирования.

Специалист по пользовательским приложениям (end-user applications specialist). Основная задача – это проверка и подтверждение data usability (применимости массивов данных хранилища) в конечных приложениях пользователя. Проще говоря, это проверка того – поступают ли необходимые данные из хранилища в конечное приложения пользователя и годятся ли они в конечном виде для проведения аналитических исследований. Модельные навыки: большой опыт работы в конечных приложениях пользователя, связанных с аналитическими исследованиями.

Разработчик (программист, development programmer) занимается разработкой приложений и утилит, которые используются в процессе проектирования и создания программного обеспечения хранилищ данных. Отдельно отметим, что это не пользовательские приложения, а приложения, которые будут использоваться группой разработки в ходе реализации проекта. Модельные навыки: опыт программирования системных утилит и приложений, опыт проектирования баз данных.

Управляющий программы обучения (lead trainer) координирует программы обучения конечных пользователей хранилища данных. Модельные навыки: навыки педагогики и обучения IT-специалистов, хорошо развитые организаторские навыки, расширенные познания в человеческой психологии, опыт разработки программ и курсов обучения.

2.2. Ключевые точки успеха проекта хранилища данных. Тупиковые сценарии проектов

Далее будут рассмотрены особенности успешных и неуспешных проектов хранилищ данных. Эти закономерности удалось выявить и оформить в виде практик вследствие того, что проекты хранилищ данных, это сравнительно редкие проекты, и практически все случаи их реализации известны. Суммируя результаты работы проектных групп со всего мира, удалось выявить некоторые закономерности, которые, несомненно, будут полезны руководителям последующих проектов.

Начнем с перечисления и описания ключевых точек успеха проекта хранилища данных. Без должного внимания этим процессам и показателям

проекта, группа проекта существенно увеличивает риск недостаточного качества проекта, а, в некоторых случаях и его неудачи.

Масштабирование и правильная оценка потенциала роста хранилища данных. Необходимо четко понимать, что после релиза, количество пользователей хранилища и количество запросов к нему будет расти в геометрической прогрессии, поэтому, при создании первичной модели оценки нагрузки на серверы хранилища нужно оценивать не текущие параметры нагрузки, а планируемые, причем с перспективой на среднесрочный период эксплуатации.

Реалистичные ожидания. В первую очередь, это касается заказчиков проекта хранилища данных. В краткосрочной перспективе эксплуатации хранилища данных никаких серьезных изменений в качестве бизнес-процессов, которые поддерживаются этим хранилищем вероятнее всего не будет. Также вероятно, что сотрудники сперва весьма неохотно будут использовать потенциал хранилища данных в своей операционной работе, больше доверяя традиционных средствам хранения и обработки данных. Отдачи от внедрения следует ожидать в среднесрочной и долгосрочной перспективе.

Внимание к моделированию измерений. Вам уже известно, что изменения в реляционных базах данных в ходе их эксплуатации «стоят» очень дорого с точки зрения трудозатрат, времени и рисков. Для страховки от этого риска следует больше внимания уделять построению логических моделей данных на этапе их проектирования. Учитывая более комплексную природу хранилищ данных разумно предположить, что изменения в ходе эксплуатации будут обходиться организации еще дороже. Предварительное моделирование измерений и определение лучшего варианта из нескольких по таблицам фактов на этапе проектирования позволит существенно сократить этот риск.

Анализ внешних потоков данных. Хранилище является агрегатной базой данных для хранения данных, приходящих из принципиально разных по своей структуре источников, – это и плоские файлы, и данные из базы данных организации и данные из баз данных контрагентов, и данные, как результат бенчмаркетинговых исследований и т.д... Тщательное изучение всех возможных источников данных существенно упростит управление ETL-процессами в ходе загрузки данных и настройку утилит для очистки данных, уже попавших в хранилище.

Внимание обучению конечных пользователей и администраторов. Если до реализации проекта хранилища данных, организация отдавала предпочтение традиционным способам хранения данных (реляционные базы данных), то

очевидно, что у конечных пользователей и администраторов (даже самых опытных), на начальном этапе использования хранилища будут проблемы на уровне языка программирования запросов к хранилищу и даже конечных программных средств, которые будут работать с данными из этого хранилища. При формировании плана обучения обязательно следует обратить внимание и на опытных пользователей аналитических приложений и специалистов по базам данных.

Согласованная бизнес-оценка используемого для хранилища стека технологий. Хранилище данных не может существовать само по себе. Для его нормального функционирования потребуется множество программных продуктов, которые способны обеспечить передачу, подготовку, очистку данных, выгрузки из хранилища, а также проведение мероприятий, связанных с аналитической обработкой полученной информации. Все эти технологии, вместе с самим хранилищем должны оцениваться комплексно (как неотъемлемая часть проекта), как одно большое решение в рамках проекта хранилища данных.

Высокая степень вовлеченности конечного пользователя в проект разработки хранилища. Это позволит добиться более лояльного отношения пользователей к конечному продукту и снизить промежуток времени, до принятия готового хранилища данных в операционную работу. Также, постоянный контакт с конечными пользователями чрезвычайно важен для хорошего результата не только в реализации функциональных требований к проекту, но и высоких эргономических, нефункциональных показателей.

Сперва – архитектура, затем – методология и в конце – инструменты. Это оптимальная последовательность решения задач на начальном этапе планирования проекта хранилища данных.

Проект хранилища данных должен включать в себя все этапы жизненного цикла продукта, включая этап его эксплуатации на длительный горизонт планирования. Хранилище постоянно должно находиться в состоянии оптимизации, настройки, модернизации. Только так можно максимально продлить срок эффективной эксплуатации дорогостоящего хранилища данных.

Смена фокуса на проектирование запросов. В отличие от реляционной базы данных, на этапе программирования элементов хранилища внимание уделяется не транзакциям (они будут выполняться особым, отличным от реляционного способом), а запросам. Это связано с тем, что зачастую аналитические запросы, ввиду своих особенностей, не повторяются (аналитикам постоянно надо искать новые тенденции, тренды, закономерности) и единственные актуальные транзакции для хранилища данных связаны с

добавлением в него новых массивов данных.

Критическое отношение к источникам данных. Не должна стоять задача: «забрать в хранилище все возможные данные». Это приведет лишь к мусорной помойке, вместо полезного элемента IT-архитектуры предприятия. Загружаться должны только действительно нужные данные, преимущественно из надежных источников и потоков.

Любые существенные отклонения от ключевых точек успеха приводят к проблемам в ходе реализации проекта хранилища данных, а в дальнейшем – к существенным неудобствам и затратам в ходе его эксплуатации. Типовые случаи тупиковых сценариев проектов хранилищ данных показаны на рис. 9.

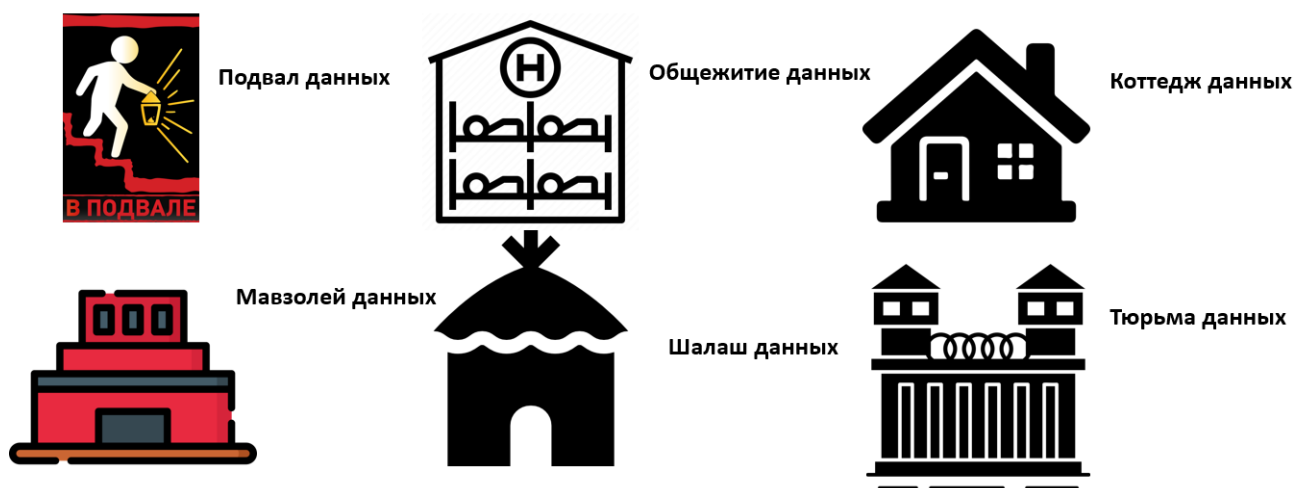


Рисунок 9. Типовые тупиковые сценарии проектов хранилищ данных

Рассмотрим приведенные сценарии подробнее.

Подвал данных. В этом случае команда проекта недостаточно внимательно относится к исследованию источников данных, предлагаемых заказчиком. Как правило, принимается решение включить все доступные источники. В итоге, группа разработки сталкивается с серьезными проблемами в ходе разработки ETL-процедур, в попытках синхронизировать изначально плохо согласованные друг с другом, а зачастую и недостоверные данные. Проблемы, связанные с источниками, сперва приводят к тому, что данные в хранилище имеют весьма плохое для проведения аналитики качество, в результате чего конечный пользователь предпочитает ими никогда не пользоваться, «запирая» их в подвал, в котором никто никогда не бывает.

Общежитие данных. На этапе планирования решено сэкономить и в качестве базовой модели для построения хранилища будет использована legacy

(прошлая, заменяемая) физическая модель базы данных. Группа проекта сознательно пропускает этап сбора требований и в итоге, логично приходит к той же модели, с ее недостатками, что были и в legacy модели хранения данных.

Коммедж данных. В ходе разработки, группа проекта уделяет внимание созданию постоянных витрин данных на основании запросов от конечных пользователей-аналитиков. В итоге, выход хранилища представляет собой строго определённый тематикой исследований набор витрин данных. При это сильно фрагментируются данные, находящиеся в хранилище, что существенно затрудняет решение задач, выходящих за пределы ожидаемых при постановке задач на проектирование хранилища массивов данных.

Мавзолей данных. На этапе проектирования хранилища заказчик так и не определился, в рамках каких процессов будет использоваться хранилище данных. «Это современная технология, она обязательно пригодится», - думал заказчик. В результате, после релиза получается ситуация, когда аналитики продолжают пользоваться традиционными источниками данных для проведения исследований и совершенно не совершают попыток перейти к взаимодействию с новым хранилищем данных. В итоге получается дорогое и бессмысленное IT-решение в дополнительную нагрузку ложится на организацию.

Шалаш данных. Выполнено сотрудниками, не имеющими ни практических, ни теоретических знаний о базовых принципах построений хранилищ данных, с полным игнорированием ключевых точек успеха проекта. В итоге получается забалованная, никому не нужная свалка несогласованных данных.

Тюрьма данных. В целом хорошо спроектированное хранилище данных, но, в ходе проекта, команда проекта редко взаимодействовала с конечными пользователями. В итоге пользователи, испытывая проблемы с затрудненным и неудобным доступом к данным отказываются от использования хранилища в пользу традиционных способов работы с данными.

2.3. Трехмерная модель хранения данных. Понятие Informational Package (информационный пакет)

Конечная цель функционирования хранилища данных – хранение, сбор и передача данных в службы аналитики компании. Именно особенности подготовки данных для проведения процедур статистического, математического анализа, машинного обучения и пр., и определяют архитектуру и характер эксплуатации хранилища. Большинство даже самых простых регулярных аналитических исследований требуют более чем двухмерную модель визуализации результата (редко когда можно обойтись двумерным линейным

графиком или гистограммой в аналитике и прогнозировании результатов бизнеса организации). Самой распространенной аналитической моделью для бизнес-исследований является так называемое *трехмерное бизнес-измерение*. По своей сути – это куб, содержащий внутри себя множество значений, по периметру которого на ребрах находятся различные возможные для имеющихся значений измерения (шкалы). Для лучшего понимания, приведем простой пример. «Золотой стандарт» базовой бизнес-аналитики – это анализ продаж по трем измерениям: что продавалось (продукт), какой срок продаж интересует (время) и место, где осуществлялись продажи (пространство или география). Структура куба для этого исследования с примерами приведена на рис. 10.

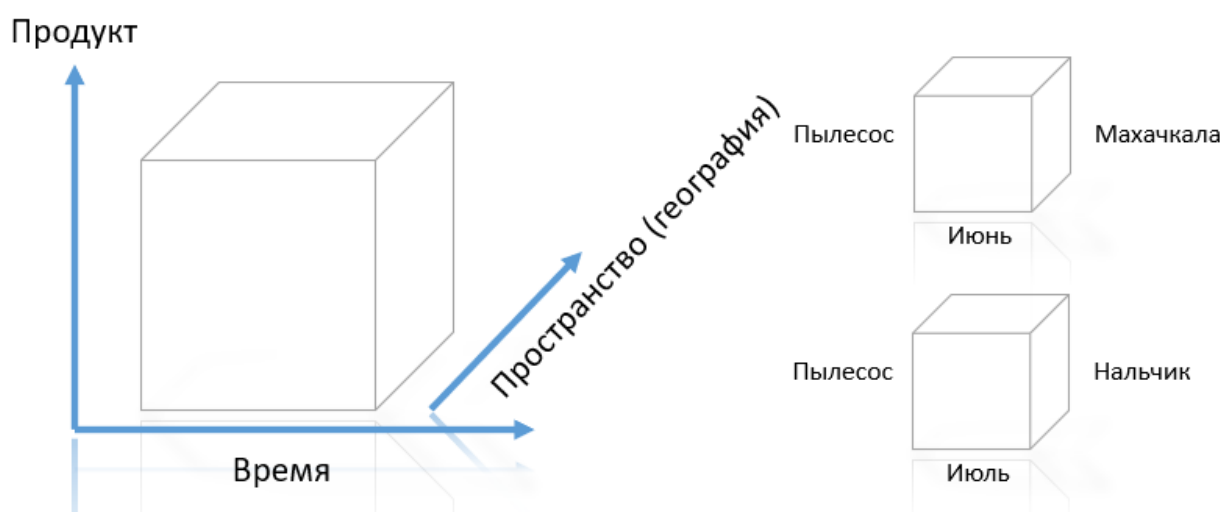


Рисунок 10. Простое трехмерное бизнес-измерение (куб)

Конкретные значения, определенные по ребрам куба в ходе процесса анализа в результате, формируют срез данных, который является ответом на вопрос задачи. Так, в приведенном примере исследователь получит срез данных по июньским продажам пылесосов в Махачкале и по продажам тех же пылесосов в июле, но уже в Нальчике.

Очевидно, не всегда задачи, стоящие перед аналитиками данных ограничены всего тремя измерениями (ребрами куба). Для их решения требуется построение большего количества измерений (векторов). Модель, которая получается в итоге, называется *гиперкуб* (многовекторные измерения). Каждый сформированный гиперкуб дает исследователю возможность получать срезы данных, как комбинацию разных интересующих его факторов в любой последовательности, используя при этом одно и то же пространство данных гиперкуба. На рис. 11 показаны несколько примеров гиперкубов для разных направлений хозяйственной деятельности организации.

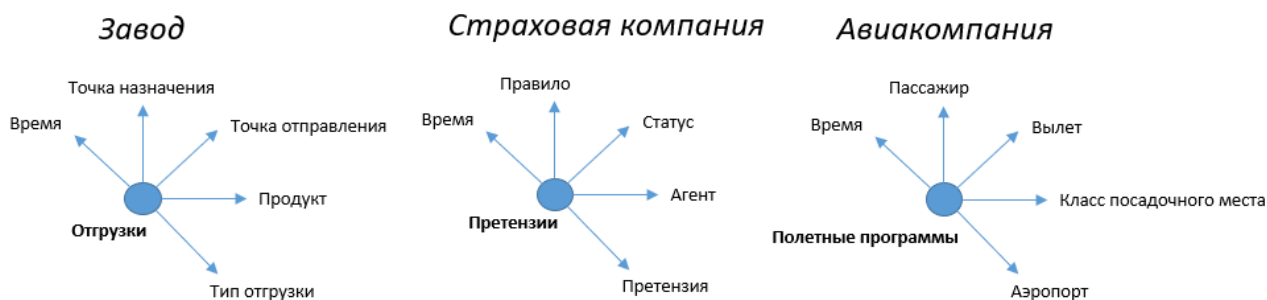


Рисунок 11. Примеры гиперкубов для разных организаций

Рассмотрим варианты гиперкубов из приведенного примера. Жирным шрифтом выделены данные из внутреннего пространства куба (как правило, это математически значимые значения, получаемые в ходе основной операционной деятельности организации). Простыми словами – это количественные данные по отгрузкам для завода, количественные данные по претензиям клиентов для страховой компании и количественные данные по полетным программам для авиакомпании. Обычным шрифтом в примерах показаны измерения, которые могут использоваться в качестве ребер гиперкуба при различных аналитических исследованиях. Так, аналитик может, наложив на пространство данных значения измерений **Время** и **Продукт**, получить срез данных по продуктам, отгруженным со склада готовой продукции завода. Отметим, что непосредственно прогнозирование не является задачей для хранилища данных и после формирования нужного среза данных, он отправляется в соответствующие программные средства аналитики (SPSS, Data Lens, R Studio и др.).

Пространство значений гиперкуба называется **измеряемыми фактами**, а пространства измерений, их окружающие называют **измерениями**. Учитывая тот факт, что измеряемых фактов может быть несколько (для завода, например, это не только отгрузки, но и передача на хранение, поставка материалов для производства, данные о производстве и т.д.), конечная модель хранилища данных может быть весьма комплексной и запутанной. Для того, чтобы существенно упростить процесс моделирования хранилища на стартовом этапе проекта его разработки, применяются таблицы с неполностью детерминированными требованиями (неформализованные спецификации), которые называются **информационные пакеты** (informational package). Эти пакеты представляют собой модель, определяющую значимые факты и измерения для планируемых к реализации в хранилище гиперкубов в компактном табличном виде. Заголовком информационного пакета является перечисление требуемых фактов, столбцы информационного пакета – названия участвующих измерений, а значения столбцов, – возможные свойства измерений.

Шаблон такой таблицы приведен на рис. 12.

Временные периоды	Локации	Продукты	Возрастные группы	...	...
Год	Страна	Класс	Группа 1		
...	...	...	...		
...	...	...	...		
...	...	...	...		
Измеряемые факты: прогноз продаж, оперативная информация о продажах					

Предметная область информации: Анализ продаж

Рисунок 12. Шаблон информационного пакета хранилища данных

Перечисление планируемых к сбору и хранению фактов обычно оформляется в нижней части пакета (прогноз продаж, данные о продажах и т.д.). Столбцы Локации, Продукты, Продавцы, Покупатели, Контрагенты и др. формируют структуру измерений, которые будут окружать факты и станут дискриминаторами в ходе процедур извлечения срезов данных для анализа из хранилища. Каждое измерение – это таблица, столбцы которой, это значения столбцов из информационного пакета. Например, столбцами измерения Временные периоды могут стать Год, Квартал, Месяц, Неделя, День и т.д...

Далее, приведем два развернутых примера информационных пакетов, созданных для изучения продаж автопроизводителя (рис. 13) и заселенности номеров отеля (рис. 14).

Время	Продукт	Тип оплаты	Демография покупателя	Дилер	...
Год	Название модели	Тип транзакции	Возраст	Название	
Квартал	Год модели	Длительность	Пол	Город	
Месяц	Дизайн	Процент	Уровень дохода	Страна	
Дата	Линейка продукта	Агент	Семейный статус	Флаг «одного бренда»	
День недели	Категория продукта		Количество автомобилей	Дата открытия	
День месяца	Цвет снаружи		Собственность		
Сезон	Цвет внутри				
Флаг праздников					
Измеряемые факты: текущая цена, полная цена, надбавка дилера, кредиты дилера, первый платеж, текущие платежи, финансы...					

Рисунок 13. Информационный пакет «Продажи автопроизводителя»

Время	Продукт	Тип оплаты	Демография покупателя	Дилер	...
Год	Название модели	Тип транзакции	Возраст	Название	
Квартал	Год модели	Длительность	Пол	Город	
Месяц	Дизайн	Процент	Уровень дохода	Страна	
Дата	Линейка продукта	Агент	Семейный статус	Флаг «одного бренда»	
День недели	Категория продукта		Количество автомобилей	Дата открытия	
День месяца	Цвет снаружи		Собственность		
Сезон	Цвет внутри				
Флаг праздников					
Измеряемые факты: текущая цена, полная цена, надбавка дилера, кредиты дилера, первый платеж, текущие платежи, финансы...					

Рисунок 14. Информационный пакет «Заселенность номеров отеля»

Совокупность информационных пакетов по всем возможным «темам» аналитики, на основании срезов, - своего рода техническое задание на проектирование логической модели хранилища данных. О принципах построения непосредственно хранилища будет рассказано в следующей главе.

2.4. Вопросы для самостоятельного изучения по теме

(кейс) Вы – аналитик проектной команды, задачей которой является создание хранилища данных для некоторой компании (например, определенная предметная область исследования диссертации). Составьте один или несколько Informational package для вашей темы (рис. 12, 13, 14).

Отчет оформить в шаблоне документа (форма документа доступна в группе предмета и будет предложена преподавателем отдельно).

2.5. Тестовые задания для самопроверки

1. Специалист команды проекта хранилища данных, ответственный за построение ETL пайплайнов, это:

- А) ведущий архитектор;
- Б) инженер данных;
- В) специалист по трансформации данных;
- Г) специалист по качеству данных;
- Д) разработчик.

2. Специалист команды проекта хранилища данных, ответственный за проверку данных в хранилище (верификацию):

- А) ведущий архитектор;
- Б) инженер данных;
- В) специалист по трансформации данных;
- Г) специалист по качеству данных;
- Д) разработчик.

3. В тем проблема тупикового сценария хранилища данных «Подвал данных»:

- А) качественные данные не попадают в хранилище;
- Б) данные в хранилище плохо структурированы;
- В) данные в хранилище бесполезны для конечного пользователя;
- Г) доступ к данным со стороны конечного пользователя затруднен.

4. В тем проблема тупикового сценария хранилища данных «Мавзолей данных»:

- А) качественные данные не попадают в хранилище;
- Б) данные в хранилище плохо структурированы;
- В) данные в хранилище бесполезны для конечного пользователя;
- Г) доступ к данным со стороны конечного пользователя затруднен.

5. В тем проблема тупикового сценария хранилища данных «Тюрьма данных»:

- А) качественные данные не попадают в хранилище;
- Б) данные в хранилище плохо структурированы;
- В) данные в хранилище бесполезны для конечного пользователя;
- Г) доступ к данным со стороны конечного пользователя затруднен.

6. Пространство значений гиперкуба OLAP называется:

- А) таблица измерений;
- Б) таблица фактов;
- В) реляционная таблица;
- Г) информационный пакет.

7. Возможные названия граней гиперкуба OLAP содержит:

- А) таблица измерений;
- Б) таблица фактов;
- В) реляционная таблица;
- Г) информационный пакет.

8. Информационный пакет хранилища данных, это:

- А) таблица с требованиями к хранилищу;
- Б) описание входных источников данных;
- В) описание витрин данных хранилища;
- Г) модель процесса сетевого обмена пакетами хранилища и клиента.

3. ШАБЛОНЫ ПРОЦЕССИНГОВОЙ АРХИТЕКТУРЫ ХРАНИЛИЩ ДАННЫХ. АРХИТЕКТУРА ХРАНИЛИЩА ДАННЫХ «ЗВЕЗДА». АРХИТЕКТУРЫ ХРАНИЛИЩ ДАННЫХ НА ОСНОВЕ HDFS

3.1. Паттерны «процессинговой» архитектуры хранилищ данных.

Достоинства и недостатки

Для начала поговорим о вариантах построения «вычислительной» или «процессинговой» архитектуры хранилища данных. Поскольку размер и сложность такого программного продукта, как хранилище данных чаще всего весьма высоки, в процессе проектирования встает нелегкий выбор архитектуры, на базе которой будет построен конечный вариант решения. Перед архитектором стоят две ключевые задачи: выбрать правильное «железо» и инфраструктурные элементы и правильно построить само хранилище из его элементов. Правильный выбор «железа» и инфраструктуры обеспечивают паттерны «процессинговой» архитектуры.

Можно выделить три основных варианта «процессинговой» архитектуры хранилищ данных, каждый из которых имеет свои достоинства и недостатки, и, соответственно – свои условия применения. Эти варианты будут описаны далее.

Архитектура SMP (Symmetric Multiprocessor) или симметричные мультипроцессоры. Сравнительно простая в реализации архитектура, где большое количество процессоров обеспечивают вычисления, при этом используя мощности общей оперативной и физической памяти через общую шину. Схема архитектуры показана на рис. 15.

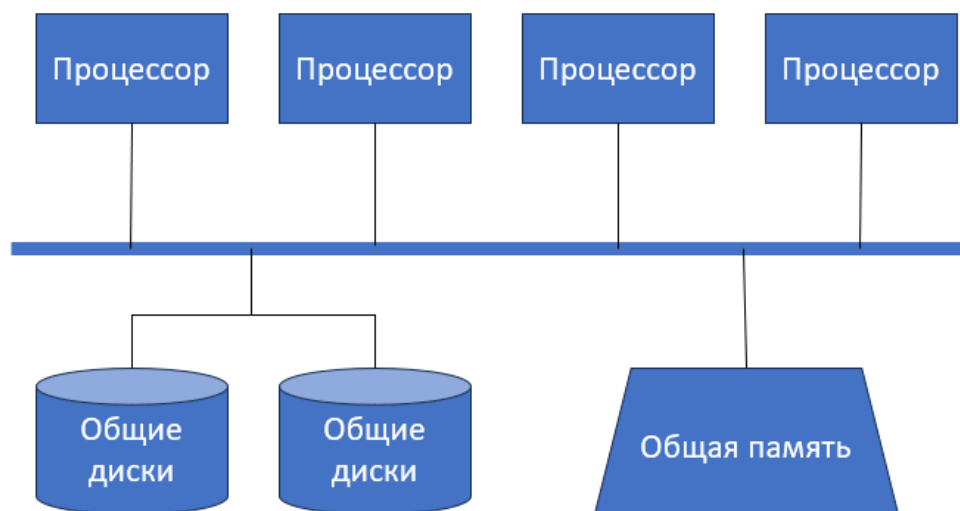


Рисунок 15. Шаблон архитектуры SMP

Особенности архитектуры:

- архитектура с самым простым принципом распределения – все распределено поровну;
- каждый процессор имеет полный доступ к общей памяти через общую шину;
- взаимодействие между процессорами происходит в общей памяти;
- взаимодействие между процессорами происходит в общей памяти.

Плюсы архитектуры:

- хорошо изученная и проверенная технология;
- высокий уровень параллелизма;
- хорошо балансируется нагрузка на память и процессоры;
- горизонтальная масштабируемость за счет увеличения количества процессоров;
- простой дизайн модели, легко администрировать.

Ограничения архитектуры:

- ограничения по памяти, рано или поздно, при постоянном расширении объема хранилища данных упрутся в потолок возможностей оперативной памяти и физической памяти сервера;
- ограничения на стороне I/O, шины, процессоров (потолок возможностей «железа»);
- примитивная архитектура (один компьютер с большим количеством процессоров).

Учитывая перечисленные выше свойства, целью применения данной процессинговой архитектуры являются хранилища данных с минимальными требованиями к параллелизму процессов (не так много одновременных обращений к серверу), а ожидаемый размер этого небольшого и простого хранилища данных от 200 до 300 гигабайтов.

Архитектура MPP (Massive Parallel Processing) или массивно-параллельная архитектура. В отличие от архитектуры SMP, процессоры тут разделены друг от друга и каждый имеет свою физическую и оперативную память. Каждый процессор, локализованный с памятью, называется узлом (node) и общается с остальными процессорами специальными коммуникационными каналами. Схема этой архитектуры показана на рис. 16.

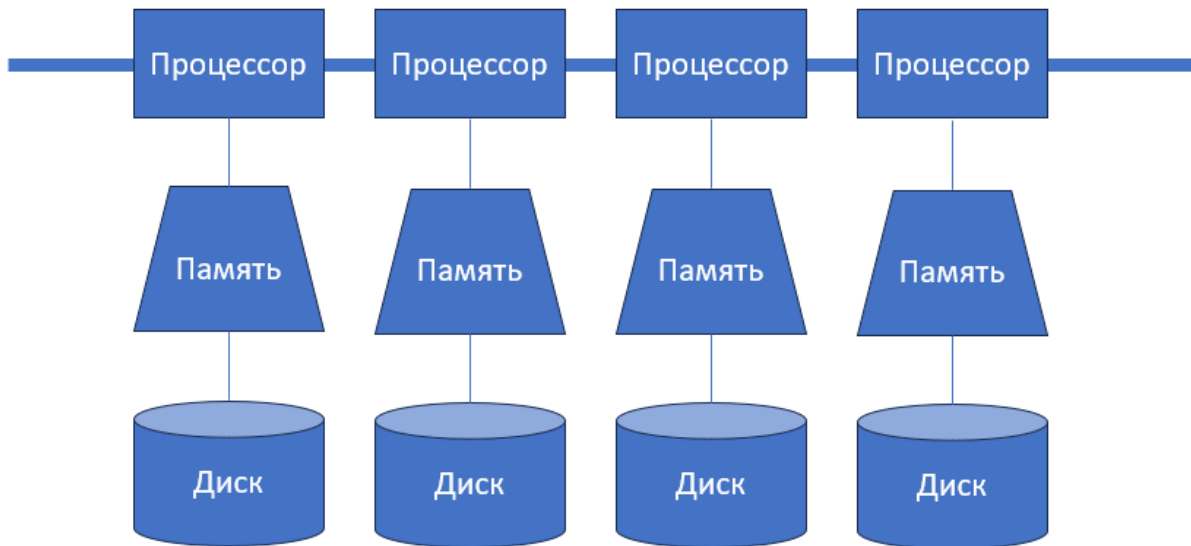


Рисунок 16. Шаблон архитектуры MPP

Особенности архитектуры:

- в архитектуре нет общих (shared) элементов;
- приоритет дисковому доступу, а не доступу к памяти;
- доступ к таблице на диске управляется тем процессором, что подключен к этому диску;
- коммуникация между узлами осуществляется диалогом между процессорами.

Плюсы архитектуры:

- обладает высокой степенью масштабируемости (но не без проблем);
- вероятный сбой ограничивает работу лишь в вышедшем из строя узле;
- минимальная стоимость затрат в расчете на узел.

Ограничения архитектуры:

- требуется жесткая настройка разделения данных, то есть – серверу хранилища надо четко указать на каком узле хранить какой массив данных, что даже с учетом автоматизации, - весьма трудоемкая процедура с большим количеством ошибок;
- ограниченный доступ к диску;
- ограниченные возможности балансировки нагрузки, фактически их отсутствие.

Учитывая перечисленные выше свойства, целью применения данной процессинговой архитектуры являются хранилища данных с обслуживаемым объемом данных до терабайта.

Архитектура кластеров (Clusters and Nodes). Компромиссный вариант между SMP и MPP архитектурами. Состоит из серверов-узлов (nodes), объединенных шиной (common high speed bus) или коммутатором. Пользователь хранилища «видит» не отдельные узлы, а весь кластер целиком. Схема этой архитектуры показана на рис. 17.

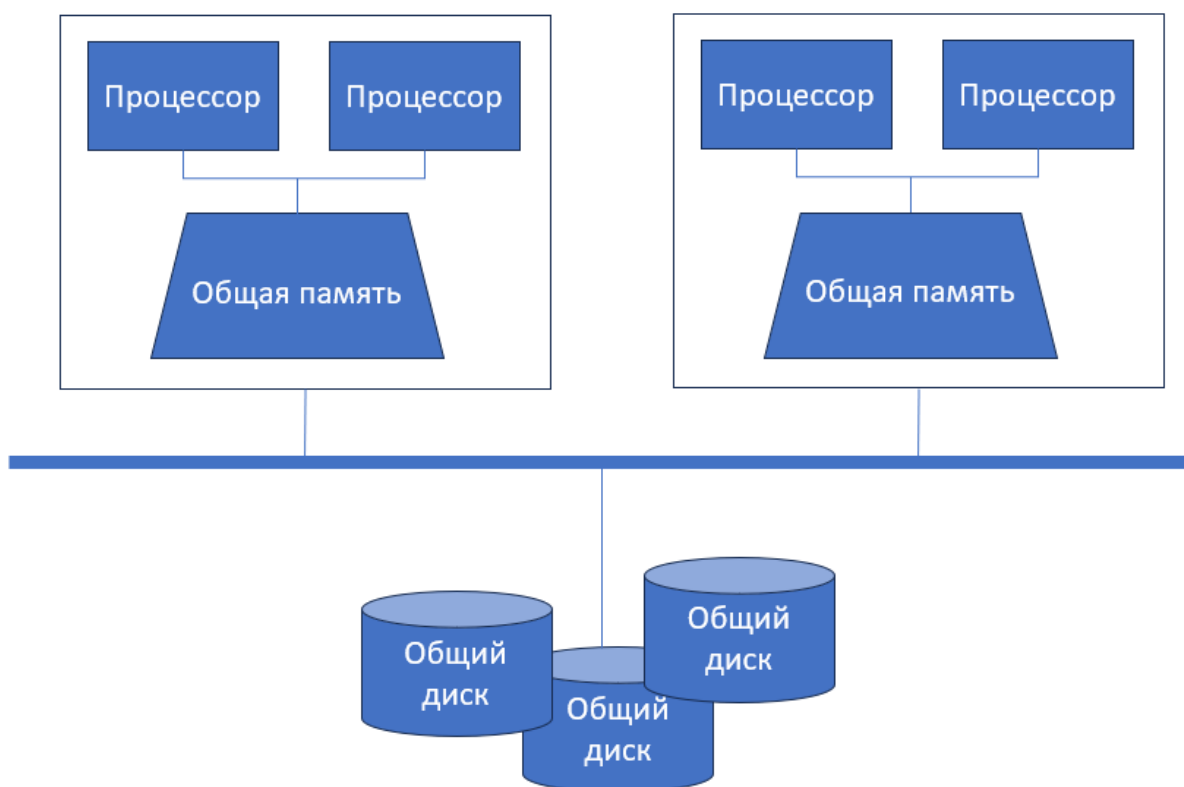


Рисунок 16. Шаблон архитектуры кластера

Особенности архитектуры:

- память не делится среди узлов, доступ есть только внутри узла;
- каждый узел имеет доступ к общему набору дисков;
- архитектура представляет собой кластер, состоящих из нескольких узлов.

Плюсы архитектуры:

- высокая степень доступности данных, все данные доступны, даже если узел выйдет из строя;
- концепция «одной базы данных» (для пользователя содержимое всех узлов кластера будет видно, как единое целое, как одно хранилище данных);
- удобная при стабильном (предсказуемом) инкрементальном росте объема

данных.

Ограничения архитектуры:

- масштабируемость (возможности по увеличению нагрузок) хранилища данных ограничивается пропускной способностью шины;
- высокие требования к «железу» и скорости работы операционной системы;
- каждый узел должен поддерживать согласованность текущего «кэша» данных (своеобразная рабочая область, содержащая используемые в данный момент времени данные).

В настоящий момент времени, это – наиболее популярный паттерн «процессинговой» архитектуры хранилища данных. Он не столь зависим от ограничений по объему данных и хорошо подходит даже для самых больших и сложных проектов хранилищ.

3.2. Архитектура традиционного хранилища данных «Звезда»

Основной принцип построения наиболее распространенной физической модели хранилища данных по архитектуре «Звезда» возник из технического требования высокого уровня: «максимальное удобство при работе с моделью для аналитиков данных». Если попытаться собрать обычный среднестатистический аналитический отчет по таблицам обычной базы данных, придется соединять данные 5-6, а иногда и большего количества таблиц. В принципе, для профессионала, нет проблемы написать скрипт в 1-2 страницы для того, чтобы справиться с этой задачей. Но что, если задача должна выполняться регулярно? Что, если задача постоянно изменяется? Но главная проблема запросов аналитических данных даже не в этом. Обычная аналитическая «рутина» в рамках купной организации – это процессы аналитического изучения данных, которые называются data drilling (буквально, просверливание данных). Для понимания этого процесса, рассмотрим схему на рис. 17. и рис. 18.

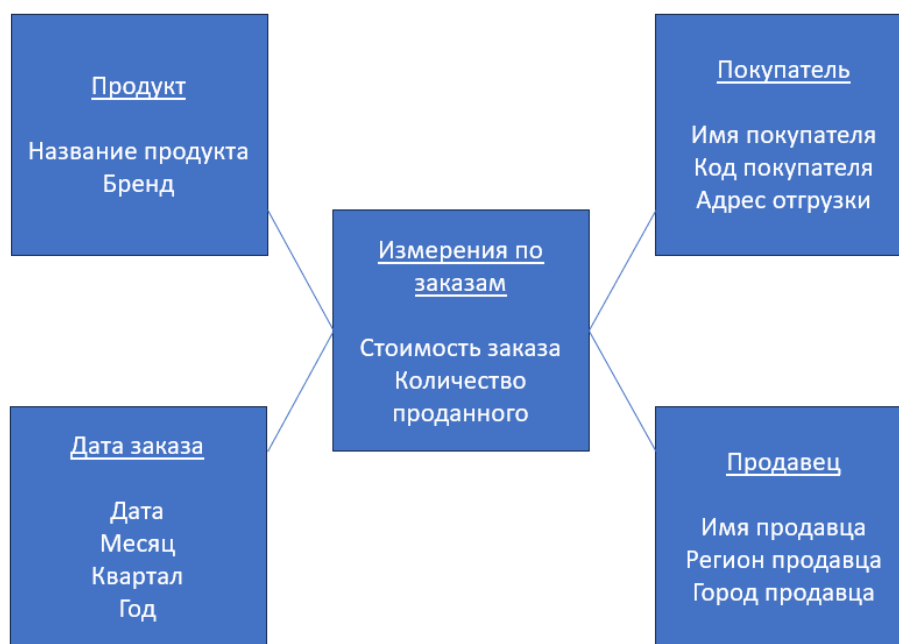


Рисунок 17. Пример структуры хранилища данных для «data drilling»



Рисунок 18. Пример процесса аналитического исследования «data drilling»

У крупной организации есть хранилище данных, одна из частей которого хранит данные о продажах компании. Пока не будем фокусировать внимание на структуре этого хранилища и ее особенностях (рис. 17). Аналитики, пользуясь этим хранилищем и прикладными средствами обработки запросов к данным проводят рутинный анализ данных с помощью серии запросов, где каждый последующий все больше «конкретизирует» данные. Аналитик (рис. 18) как бы движется с поверхности (Бренд, Год, Регион), внутрь данных (Город, Квартал, Продукт). Такое «сверление» данных вглубь – это большая часть запросов, поступающая на сторону хранилища данных. И делать это «сверление» на нормализованной реляционной модели данных весьма неудобно и трудоемко. Именно поэтому появилась модель хранилища данных типа «Звезда», где

внутренние компоненты подчиняются парадигме гиперкубов OLAP. Состоит она не из традиционных реляционных таблиц, а из таблиц фактов и таблиц измерений, которые не подчиняются условиям нормализации и базовых нормальных форм реляционной модели хранения данных. Далее, разберем такие элементы хранилища данных типа «Звезда», как таблицы фактов и таблицы измерений.

Таблица измерений содержит преимущественно текстовые описания, логически объединенные одной темой. Так, - Имя, Адрес, Номер телефона, Статус покупателя являются символьными описаниями экземпляров объекта Покупатель и, соответственно – атрибутами таблицы измерений Покупатель. У каждой таблицы измерений есть ключ, уникально определяющий строки.

Эти таблицы могут быть весьма «широкими» - до 50 атрибутов и более, где это необходимо. В зависимости от того – нормализуются ли эти таблицы или нет, схема хранилища данных определяется как «Звезда» (если таблицы измерений не нормализованы), или как «Снежинка» (если таблицы измерений нормализованы в соответствии с требованиями реляционной модели хранения данных).

В этих таблицах никогда не хранятся математически значимые данные (данные, которые можно применить в рамках математических или статистических процедур).

Для удобства процедуры data drilling (рис. 18), данные таблиц измерений часто гранулируются (год, месяц, день и т.д.) или выстраиваются в иерархии (январь, февраль, март и т.д.).

Количество экземпляров в измерении, как правило, сравнительно невелико.

Таблица фактов, напротив, содержит математически значимые данные, которые будут использоваться при решении задач аналитики данных. Если таблицы измерений, это ребра гиперкубов (рис. 10), то таблицы фактов – это их содержимое.

При построении схемы «Звезда», таблица фактов содержит внутри себя не только собственный ключ, уникально определяющий находящиеся внутри экземпляры, но и аккумулирует ключи всех измерений, связанных с этими фактами.

В одной таблице фактов могут находиться метрики с разным уровнем детализации (совокупные продажи, продажи по продуктовым линиям, продажи по продуктам, и т.д.).

Математически значимые данные, находящиеся внутри таблицы фактов,

называются мерами. В зависимости от того, какие математические процедуры можно с ними осуществлять, эти меры могут быть дегенеративные, неаддитивные, полуаддитивные и аддитивные (подробнее, в соответствующей лабораторной работе).

В отличие от таблиц измерений, таблицы фактов не «широкие» (не содержат большое количество атрибутов), а «глубокие» (содержат большое количество экземпляров).

Допустимо использование «разреженных данных». Это массивы данных в абстрактном представлении, где данные представлены не непрерывно, а фрагментами (обычный массив – это кривая, разреженный массив – это, например, гистограмма).

Если необходимо, таблица фактов может совсем не содержать фактов.

На рис. 19 показана абстрактная модель хранилища типа «Звезда», в которой демонстрируется каким образом формируются таблицы измерений, и как они логически соединяются с фактами об объектах.

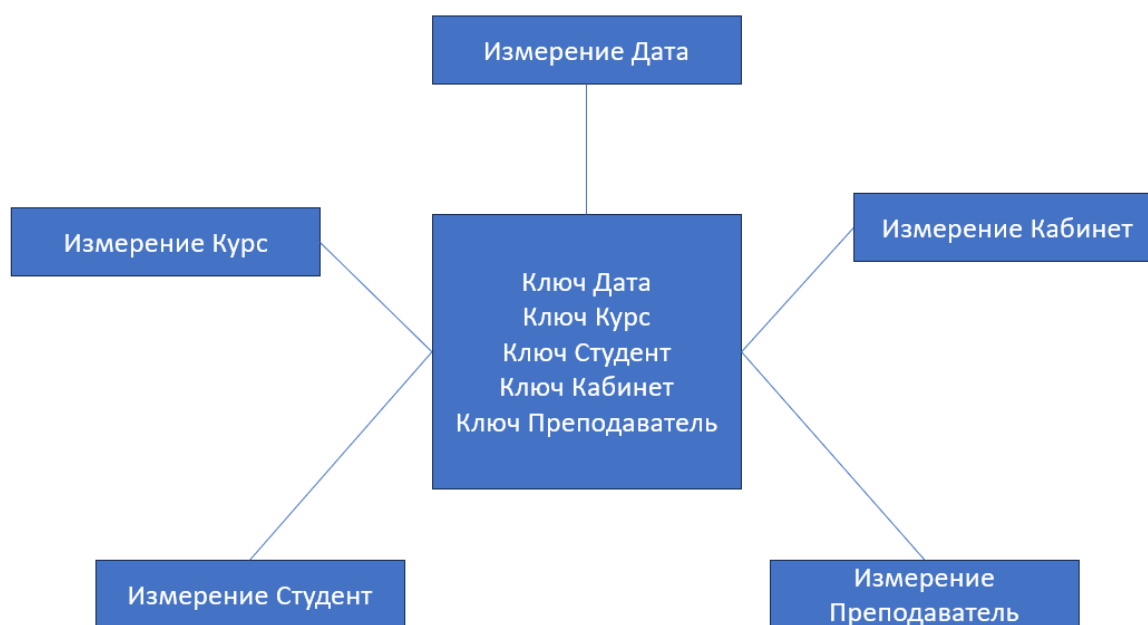


Рисунок 19. Абстрактная модель хранилища данных типа «Звезда» на уровне ключей

На рисунке видно, как таблица фактов по проведенным Курсам собирает внутри себя ключи всех измерений, которые с ней соединены. Внешне, такая структура напоминает рисунок звездочки, в связи с чем, она и получила такое название. Далее, на рис. 20-22 будет приведен ряд примеров моделей хранилищ данных типа «Звезда» для различных аналитических задач, в разных предметных областях.

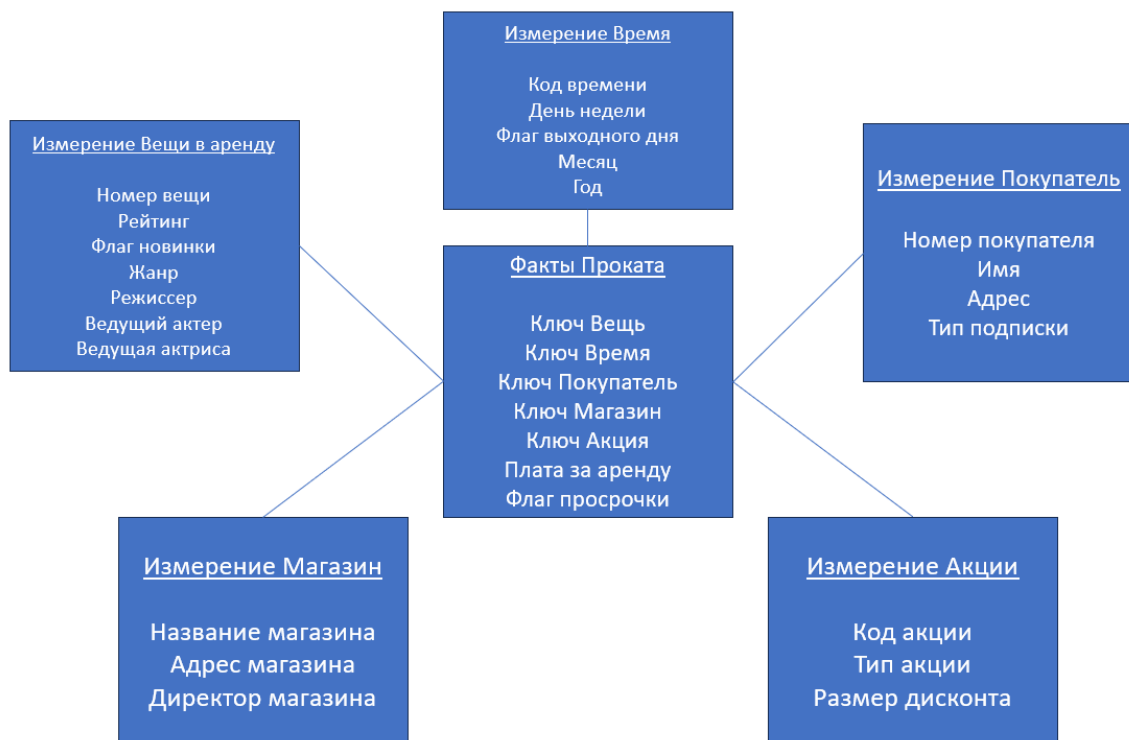


Рисунок 19. Модель хранилища данных типа «Звезда» для сети видеопроката



Рисунок 20. Модель хранилища данных типа «Звезда» для сети супермаркетов

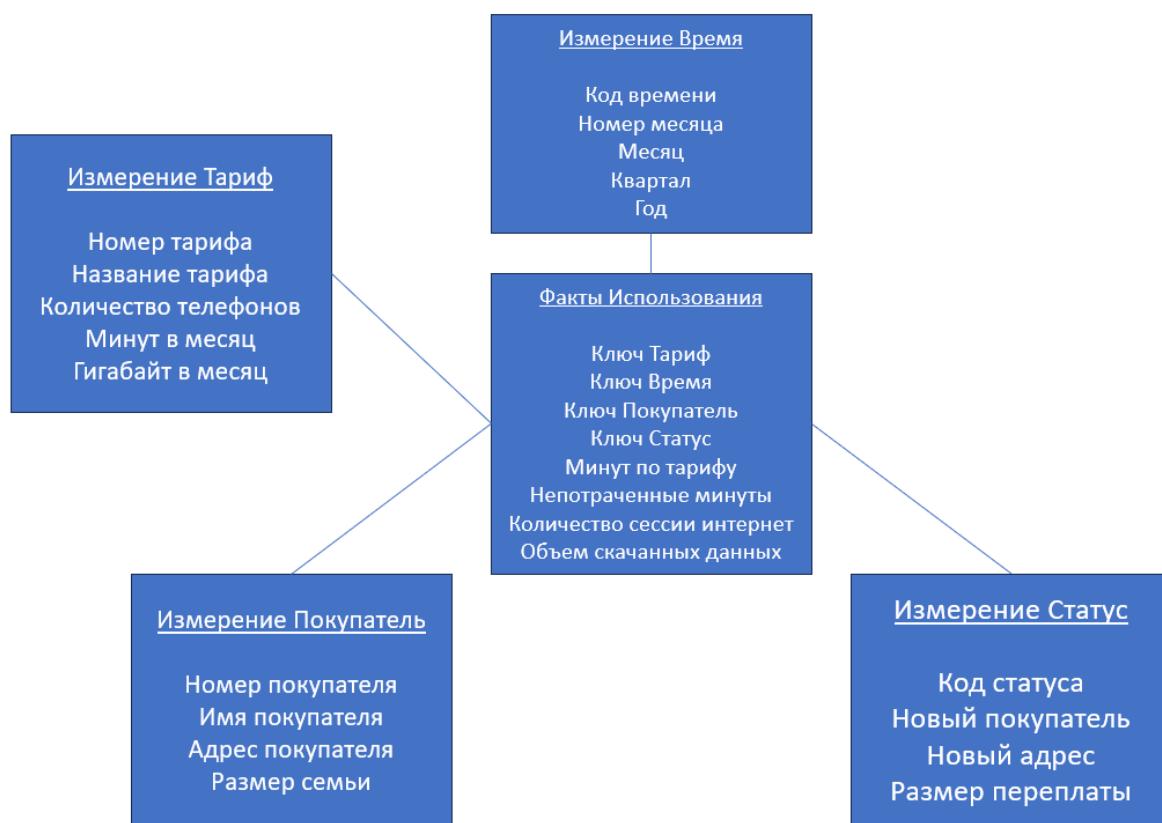


Рисунок 21. Модель хранилища данных типа «Звезда» для провайдера мобильной связи

Как видно из моделей, приведенных на рисунках, структура хранилища данных типа «Звезда» для разных предметных областей, все равно имеет общие черты и в целом схожие атрибуты для измерений и фактов. Это обстоятельство также является существенным отличием хранилищ этого типа от реляционных баз данных. Далее будут рассмотрены два типовых паттерна, характерных для хранилищ данных типа «Звезда»: «семейство звезд» и «согласованные измерения».

Концепция «семейства звезд», это более широкий и комплексный подход к построению архитектуры хранилища данных типа «Звезда». За счет создания в одной модели нескольких центров в виде таблиц фактов, структура хранилища становится более комплексной. При этом, с помощью данных из этого хранилища можно решать значительно более широкий спектр аналитических задач. В абстрактном виде, концепция «семейства звезд» показана на рис. 22.

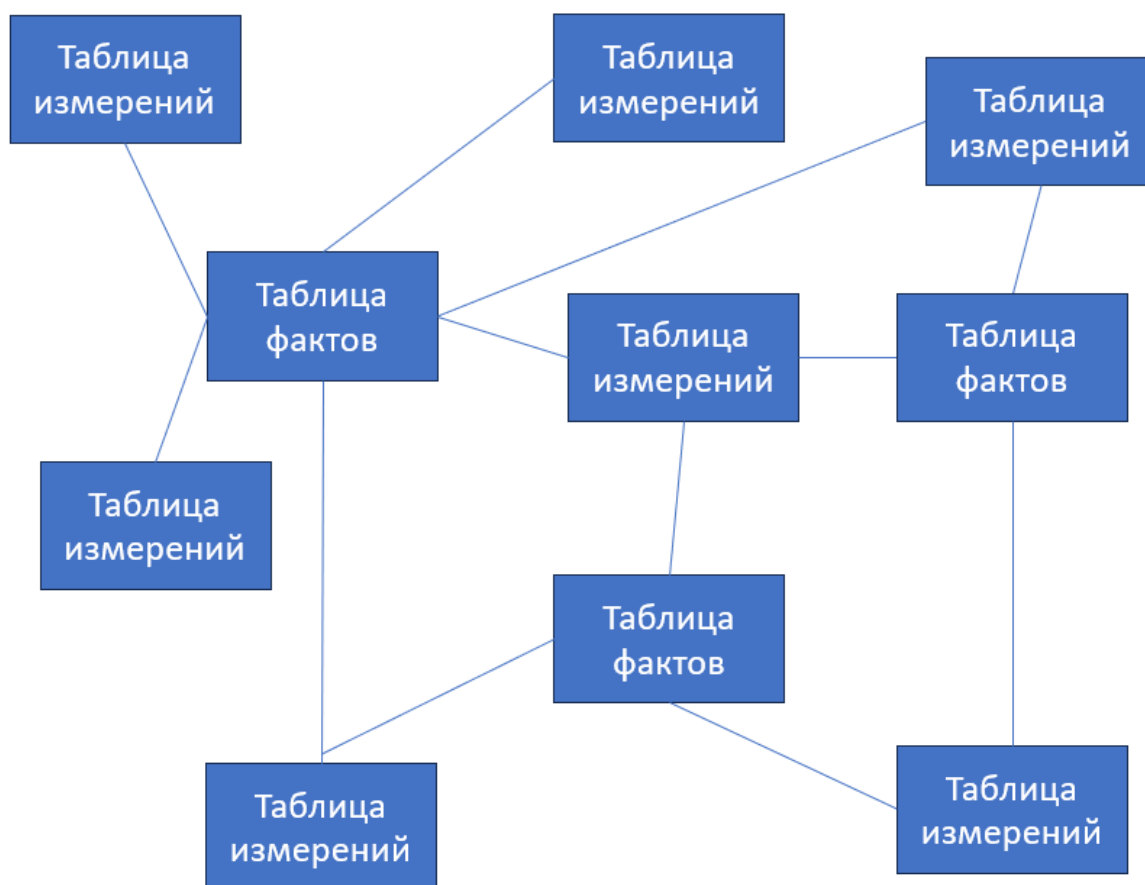


Рисунок 22. Абстрактная модель «созвездие» или «семейство звезд» хранилища данных

На рисунке видно, что в рамках одного хранилища может быть создано неограниченное количество таблиц фактов и таблиц измерений, связанных друг с другом.

Следует обратить внимание, что на рис. 22 есть измерения, которые предлагают свои данных для разных таблиц фактов. Одна из таблиц измерений связана с тремя таблицами фактов. Это стандартная ситуация для хранилища данных типа «Звезда», а такие *измерения называются «согласованными»*. Так, например, данные таблиц измерений Продукт, Покупатель, Продавец, Дата могут одновременно использоваться таблицами фактов Факты Заказа и Факты Отгрузки.

3.3. Шаблоны архитектуры хранилищ данных на основе файловой системы Hadoop (HDFS) для «Больших данных»

Средствам построения архитектуры Big Data хранилищ данных будет уделено соответствующее внимание в последующих лекциях. В рамках текущей главы разберем один из популярных технологических стеков построения кластерной архитектуры хранилища данных на основе популярной платформы

данных Cloudera (Cloudera Data Platform, CDP). Эта платформа включает в себя широкий выбор технических средств для построения хранилищ для больших данных. Образец модели кластерной архитектуры CDP показан на рис. 23.

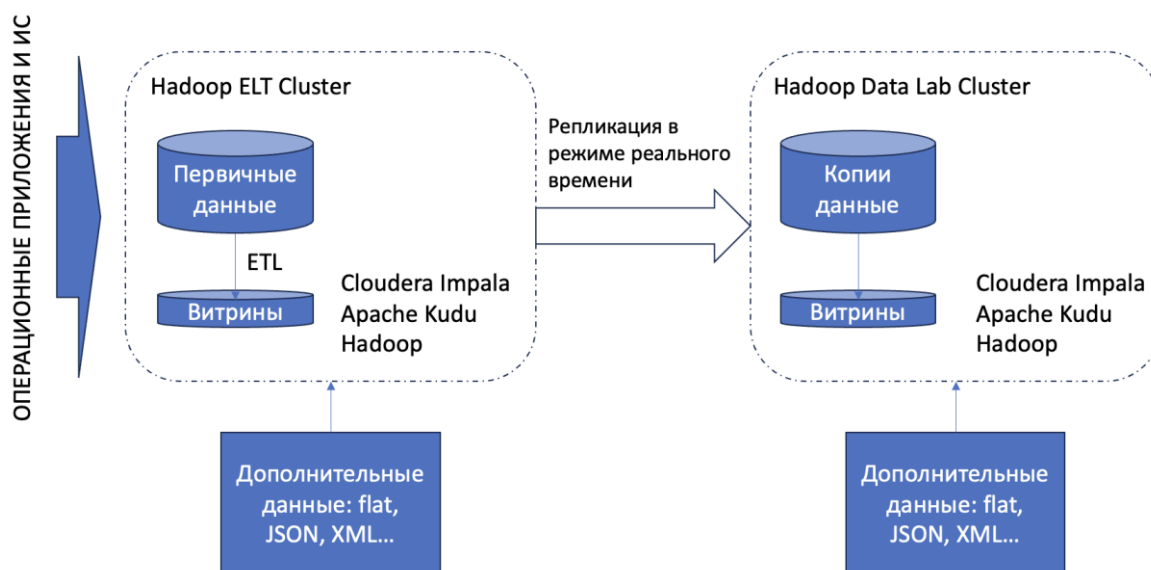


Рисунок 23. Типовая модель хранилища больших данных на базе платформы CDP

Из операционных приложений и информационных систем, информация попадает в виде первичных данных в хранилище больших данных на базе фреймворка Hadoop и HDFS (Hadoop Distributed File System). Программные средства Cloudera Impala (выполнение потоковых SQL запросов) и Apache Kudu (колоночная база данных, близкая по логике работы к реляционным) обеспечивают упорядоченность и согласованность данных. Кластер Hadoop ELT отвечает за регламентные работы по содержанию и трансформации данных перед передачей их в витрины для аналитики.

Передача данных в кластер для пользовательской аналитики (Hadoop Data Lab Cluster) осуществляется настроенной в режиме реального времени репликацией. Данный кластер с целью обеспечения максимальной эффективности и согласованности с остальными кластерами, также обеспечен ПО колоночной базы данных и средствами потокового выполнения SQL запросов. Именно в нем создаются пользовательские витрины данных, которые и используются во внешних приложениях анализа данных.

Отметим также, что внешние данные могут быть получены не только на входе в кластер ELT и затем извлечены из общего хранилища на базе HDFS, но и специально подгружены в формате плоских файлов, документов JSON и XML в случае необходимости напрямую в нужный кластер.

3.4. Вопросы для самостоятельного изучения по теме

1. Самостоятельно изучите программные продукты, входящие в платформу работы с большими данными Oracle BigData Lite. Перечислите их и кратко опишите, или изобразите технологический стек аналогично рис. 23.
2. Дайте расширенное определение терминам «Map-Reduce», «HDFS», BigData.
3. Из каких основных элементов состоит колоночная СУБД?

3.5. Тестовые задания для самопроверки

1. Для какой из перечисленных ниже архитектур хранилищ данных характерны единые физические и операционные мощности для хранения данных?
А) Clusters and nodes;
Б) SMP;
В) MPP;
Г) шарды можно создать в любой из перечисленных архитектур.
2. К какой из перечисленных ниже архитектур хранилищ данных относится модель шардирования?
А) Clusters and nodes;
Б) SMP;
В) MPP;
Г) шарды можно создать в любой из перечисленных архитектур.
3. Какой из перечисленных типов мер не относится к таблице фактов хранилища данных?
А) дегенеративные;
Б) аддитивные;
В) неаддитивные;
Г) все перечисленные меры относятся к таблице фактов.
4. Статистически значимые данные в хранилище данных помещаются в...
А) таблицы фактов;
Б) соответствующие им по смыслу таблицы;
В) таблицы измерений;
Г) индексы.

5. «Широкие» таблицы измерений характерны для хранилища данных типа...

- А) колоночная база данных;
- Б) хранилище «Звезда»;
- В) хранилище «Снежинка»;
- Г) документная база данных.

6. Нормализованные таблицы измерений характерны для хранилища данных типа...

- А) колоночная база данных;
- Б) хранилище «Звезда»;
- В) хранилище «Снежинка»;
- Г) документная база данных.

7. Основной элемент для хранения данных в хранилищах BigData это...

- А) колоночная база данных;
- Б) совокупность витрин данных;
- В) серверы-репликации;
- Г) распределенная файловая система Hadoop.

4. ПРОЦЕДУРЫ ETL (EXTRACT, TRANSFORM, LOAD)

4.1. Определение ETL процессов, сферы применения, типовая последовательность действий

Работа с данными в хранилище не ограничивается их хранением и выдачей конечным пользователям или в программы формирования аналитической отчетности. Большая часть задач решается на уровне получения и первичной обработки данных, для последующего помещения их в хранилище. В одной из первых лекций было отмечено, что одной из предпосылок к появлению такого способа хранения данных, как хранилище, стали все чаще встречающиеся разнородные потоки входящих данных. Чем крупнее организация, тем больше входных источников данных она имеет. Разные потоки данных имеют разные схемы, разное количественное и качественное выражение, а также разную степень доверия со стороны конечного пользователя. Очевидно, что в этих условиях критически важно иметь возможность получаемые данные «предобрабатывать» перед тем, как этот массив, как единое целое соединится в самом хранилище.

Для лучшего понимания, представим вполне обычную ситуацию, которая приводит к существенной разнородности поступающих на вход данных. Некоторая крупная организация занимается продажей бытовой техники. За годы своего существования она не только заработала репутацией много клиентов, но и диверсифицировала свой бизнес, в настоящее время предлагая клиенту не только развитую сеть оффлайн-магазинов (с чего все и начиналось), но и возможность изучать, выбирать и приобретать товары посредством веб-приложения интернет-магазина. При этом, с точки зрения продавцов (занимающихся аналитикой продаж), работающих в этой компании, данные, поступающие из оффлайн и онлайн магазинов однородные и, в некоторых задачах могут понадобиться в виде единого массива.

Для того, чтобы объединить эти два потока данных в рамках одного хранилища могут понадобиться недюжинные усилия: back-end оффлайн магазина с большой долей вероятности – показавшее себя годами с наилучшей стороны, но, вместе с этим «дремучее» legacy-решение, клочками модернизируемое с годами. Завязано оно на классическую строгую реляционную модель хранения данных, подразумевающую всеобъемлющую информацию о всех элементах хранения в виде метайнформации (в первую очередь – схемы таблиц). В то же самое время, интернет-магазин, ввиду своей новизны, вполне может быть построен, основываясь на современных технологиях хранения

больших данных и скидывать полученные от транзакций данные в документную poSQL базу данных, не требующую строгой схемы для своих элементов хранения.

В приведенной ситуации налицо потенциальный конфликт двух потоков вроде бы однородных, но несогласованных по своей метайнформации данных. Для того, чтобы совместить данные этих двух потоков в один массив для загрузки в хранилище, сперва требуется их «предобработать», или, как говорят специалисты по хранилищам данных, - провести процедуру ETL.

ETL, или Extract, Transform, Load – это группа последовательных процедур, направленных на извлечение, трансформацию и загрузку данных, полученных из разных источников.

В приведенном выше примере, одним из вариантов решения задачи объединения двух потоков данных может быть приведение на этапе Transform слабо согласованных данных, приходящих из документной СУБД к строгому, реляционному виду (проверить и переименовать столбцы, расставить физические реляционные типы данных, где это необходимо, удалить из столбцов несогласованные по типу данных значения или заменить их на значения по умолчанию и т.д.). После процедуры изменения данных одного из потоков и согласования их с данными первого потока, эти данные объединяются в один массив и загружаются в конечное место хранения (например, хранилище данных).

Следует отметить, что область применения процессов ETL не ограничивается только лишь хранилищами данных. Ниже, с краткими комментариями, приведены основные направления применения ETL в современных IT решениях.

Облачная миграция. Процесс переноса данных из приложений в облако называют облачной миграцией. Компании, в поисках экономии средств, расходуемых на поддержание своей IT архитектуры переносят свои данные, приложения в облачные сервисы, свои или аутсорсинговые. Как правило, такая миграция однозначно и весьма заметно сокращает нагрузку на серверы и сетевую инфраструктуру компании, позволяя освободить значительные денежные средства для решения других задач. Также, такая миграция позволяет сделать приложения более масштабируемыми и дополнительно защитить данные.

Непосредственно хранилище данных, или специфическая модель хранения данных, куда передают данные из различных источников, чтобы их можно было совместно анализировать в коммерческих целях.

Машинное обучение. Сбор, очистка, трансформация распределённых данных, используемых в процессе обучения автоматизированных интеллектуальных систем (робототехники, систем нечеткой логики, баз знаний, нейронных сетей и иных проявлений искусственного интеллекта). Само по себе машинное обучение, это метод анализа данных, который автоматизирует построение аналитических моделей.

Интеграция маркетинговых данных. Маркетинговая интеграция включает в себя перемещение всех маркетинговых данных - о клиентах, продажах, из социальных сетей и веб-аналитики в одно место, чтобы получить возможность их проанализировать. Как правило, данные полученные в результате операционной деятельности организации, бенчмаркинга интересующего сегмента рынка и деятельности штатных маркетологов довольно сильно отличаются по своей структуре (схеме). ETL используют для объединения маркетинговых данных.

Интеграция данных IoT. При работе в окружении Internet of Things суть сбора данных заключается в считывании и упорядочивании данных, собранных различными датчиками, в том числе встроенными в оборудование. Датчики могут работать как в цифровом, так и в аналоговом режиме. Процедуры ETL позволяют корректно и быстро собрать данные от разных IoT в одной базе, что позволит осуществлять мониторинг самих устройств и анализ их активности.

Репликация базы данных — данные из исходных баз данных копируют в облачное хранилище или на серверы-реплики. Это может быть одноразовая операция или постоянный процесс, когда ваши данные обновляются в облаке или на репликах сразу же после обновления в исходной базе. Реплицированные данные можно использовать как для процедур администрирования (резервное копирование, планирование развития модели хранилища и т.д.), так и для построения постоянных витрин данных.

Далее, приведем типовую последовательность шагов процедуры ETL, проводимой в рамках хранилища данных.

1. Определение данных, которые следует переместить в хранилище. На данном этапе, относительно готовой модели хранилища, или относительно технического задания к проекту разработки хранилища данных определяются объекты хранения (таблицы) и их схема (информация о столбцах и ограничениях).

2. Определение всех внешних и внутренних источников данных. После определения всех схем таблиц собирается информация об источниках данных.

Учитываются все возможные имеющиеся и перспективные источники. Также заранее определяется приоритет источников, степень доверия к ним, их активность.

3. Подготовка mapping для всех элементов данных из источников. Готовятся схемы трансформации данных для однородных источников. Главная задача – понять, какие предстоит осуществить манипуляции со схемами данных, поступающих из изученных ранее источников, чтобы конечный массив информации совпадал с разработанной схемой хранилища данных.

4. Формирование правил извлечения, трансформации и загрузки данных. В документе, помимо плана трансформации данных обычно приводят скрипты требуемой трансформации, написанные или на языке SQL (традиционные пайплайны) или на языке Python (DAG пайплайны).

5. План агрегации данных в таблицах (где это необходимо). В документе приводится информация о том, какие потоки данных будут объединены в один массив. Также как и в случае трансформации, приводятся скрипты объединения массивов данных, написанные на языке SQL.

6. Написание кода ETL процедур (SQL, интерфейс или, например Python). Процедуры содержат указатели на операции с данными, начиная с их выгрузки из источника, и заканчивая загрузкой в хранилище. Если планируется осуществлять эти процедуры многократно, добавляются средства автоматизации (код или программный интерфейс).

7. ETL для таблиц измерений и фактов. Данные, уже попавшие в хранилище после проведения ETL процедур необходимо проверить на согласованность и «почистить» от ошибок и мусора. Для этого разрабатываются специальные процедурные ETL процессы, которые будут запускаться изнутри хранилища данных.

4.2. Пайплайны и требования к процессам ETL

Пайплайном или конвейером ETL называется набор предварительно настроенных процессов и специальных инструментов, позволяющий в автоматическом режиме последовательно загружать, преобразовывать и загружать данных из распределенных источников (потоков).

Пайплайны, как правило, настраиваются и выполняются в специализированном программном обеспечении. Так, в рамках практических работ курса, нами будут рассмотрены популярные программные средства SQL Server Data Tools (SSDT), используемое для визуального моделирования пайплайны для хранилищ данных на базе MS SQL Server и Apache Airflow, более универсальное средство создания пайплайнов в языке Python.

Типовой процесс создания пайплайна ETL включает в себя 9 последовательных этапов:

- подключение к источникам данных, это могут быть «плоские» файлы, документы JSON и XML, базы и хранилища данных, API информационных систем и т.д.; при настройке подключения необходимо учитывать не только параметры входного потока данных, но и параметры конечной точки назначения потока данных – хранилища;

- настраивается процедура извлечения данных, после успешного подключения формируется массив данных (выборка) для передачи в дальнейшую обработку;

- процедура преобразования данных, как правило, включает в себя действия по применению фильтров к данным и их агрегации;

- установление соединения с местом назначения, это может быть промежуточный документ JSON или XML, промежуточная реляционная база данных или непосредственно хранилище данных;

- настройка процесса загрузки данных; именно на этом этапе происходит форматирование самих данных и изменения в их схеме, с целью синхронизации данных, полученных из разных источников;

- расписание и автоматизация работы пайплайна; на этапе определяется – как часто будет включаться пайплайн (периодичность), или какие события будут активировать запуск ETL процедуры;

- обязательно настраивается процедура обработки ошибок и протоколирование ETL процесса; как можно внимательнее следует отнестись к процессу выявления и своевременной передачи в обработку ошибок процесса, под протоколированием же понимается ведение журнала работы пайплайна во время каждой итерации его работы;

- тестирование созданного пайплайна; на тестовых наборах данных неоднократно проверяется качество работы настроенных процедур, только тщательная проверка позволит сократить количество дорогостоящих сбоев в работе одного из ключевых процессов для всего хранилища данных;

- развертывание и мониторинг, как бы качественно не был составлен процесс и как бы тщательно он не был проверен, сбои в работах пайплайнов – весьма часто встречающееся явление (недоступность одного из источников, ошибки на этапе извлечения, проблемы преобразования и загрузки и т.д.), поэтому требуется мониторинг всех пайплайнов в режиме реального времени.

Типовая схема работы пайплайна показана на рис. 24.

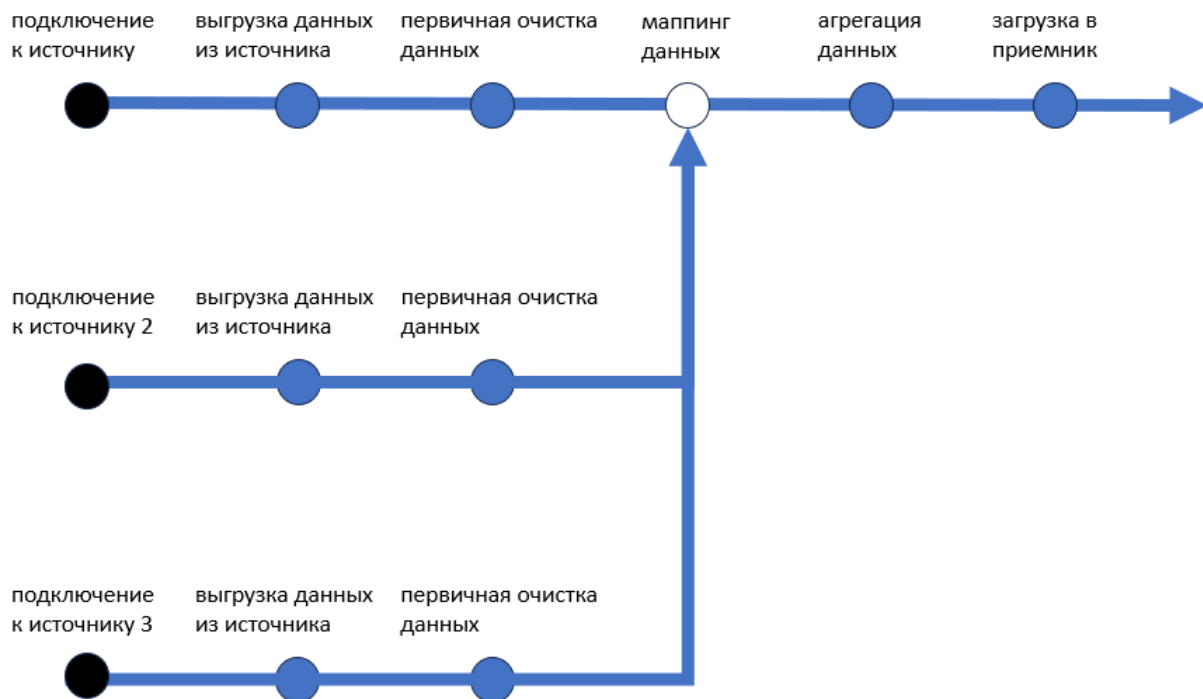


Рисунок 24. Типовая схема работы пайплайна ETL

На рис. 25 показан небольшой пайплайн, выполненный средствами программного средства SSDT (SQL Server Data Tools). Данные получают в виде потока и в случае корректности транзакции сохраняются в «плоский» файл формата *.txt. В случае ошибки транзакции, файл с данными будет удален.

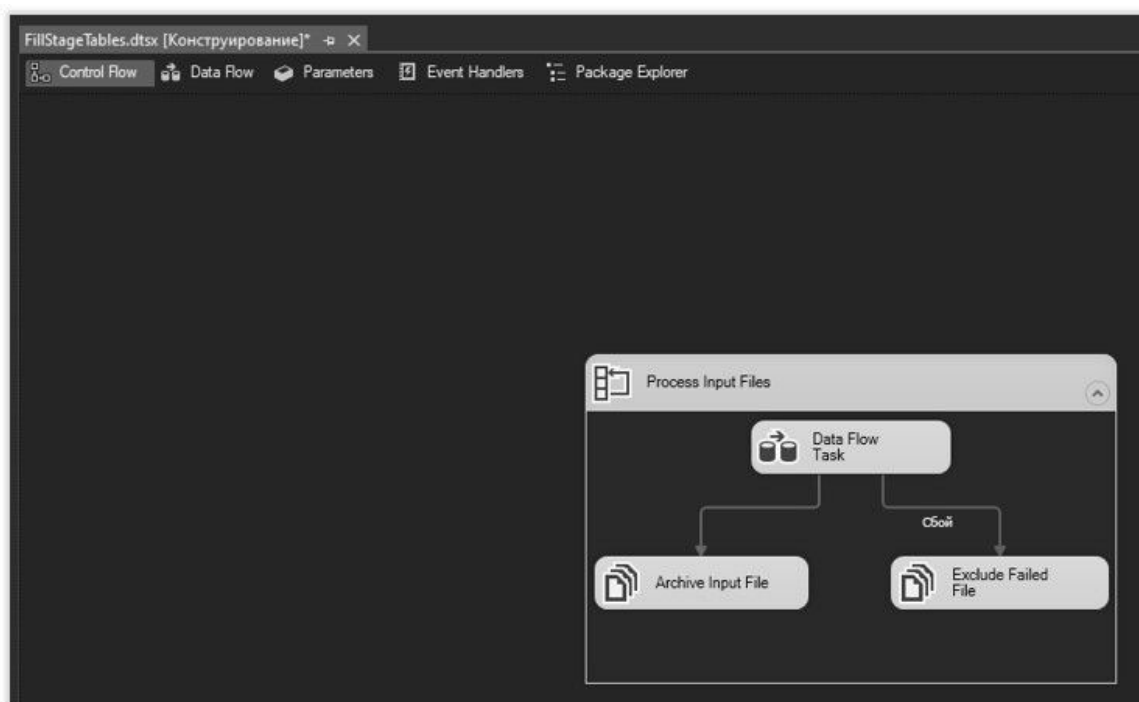


Рисунок 25. Фрагмент пайплайна SSDT

Применительно к процессу ETL, существует ряд требований ко всем ключевым этапам этого процесса. Требования считаются методическими, и строгое им следование позволит существенно сократить время разработки и тестирования пайплайна.

Требования к процессу идентификации ресурсов (extract)

Предварительное составление простого списка метрик или фактов для всех таблиц фактов в хранилище. Можно представить его в виде аналога логического словаря данных для реляционной модели хранения данных. При описании мер, следует явно указывать тип каждой меры (дегенеративная, аддитивная, неаддитивная).

Составление простого списка атрибутов измерений для всех таблиц измерений. Также, как и со словарем фактов, следует уточнять тип столбца для таблиц (имя, атрибут, пояснение).

Для каждого элемента из обоих словарей следует подобрать актуальный источник данных (или несколько, если это необходимо) из имеющихся на входе (если для одного элемента может быть несколько источников одной и той же информации, следует выбрать доверенный).

Если в одно значение хранилища приходят объединенные данные из нескольких источников (например, дата или id), необходимо сформировать правила объединения нескольких потоков данных в один. В обратном случае, сформировать правила разбиения.

Следует определить для всех потоков данных значения «по умолчанию», которые можно применять в большинстве конфликтных ситуаций в ходе исполнения процесса.

Основные задачи в ходе трансформации данных (transform)

Выбор — в ходе выполнения процесса формируется конечный массив данных (построенной на полной или фрагментарной информации из источников), который будет переноситься в хранилище данных.

Разделение/соединение данных (часто — соединение, редко - разделение).

Конверсия — большой набор задач, применяемых к данным в массивах с целью стандартизации данных из разных источников, а также повышения «понятности» и «полезности» данных для конечных пользователей.

Агрегация — добавление свойства «гранулярности» к данным в массивах (например, если имеются данные «по дням», после агрегации можно получить массивы данных «по неделям», «по месяцам» и т.д.).

Насыщение – изменение порядка, упрощение столбцов с данными в таблицах с целью более удобного их представления в конечном виде.

Требования к процессу загрузки данных (load)

Учет требований бизнеса по длительности итераций процессов загрузки данных в хранилище (например, «загрузка в течение недели готовых в исходных системах наборов данных в 40 итераций по 1 часу в нерабочее время»).

Проектирование технических справочников (автоматизированной документации, специальных журналов транзакций) в случае загрузки данных «набегающей волной» (обновление одних и тех же наборов данных в течение последовательных периодов).

Обеспечение «версионности» наборов данных за счет создания технических справочников (автоматизированной документации, специальных журналов транзакций) для ведения конфигураций и обеспечения поточности загрузок.

Деление пакетов данных по количеству исполнителей, ответственных за их заполнение (выделение «быстрых» и «медленных» пакетов данных для существенной оптимизации времени загрузки).

4.3. Техники загрузки данных в хранилище, выявление ошибок при процессе валидации данных

В зависимости от особенностей хранилища и данных, которые в нем хранятся, процедура загрузки может осуществляться в рамках одной из четырех техник загрузки: простая загрузка (load), загрузка добавлением (append), загрузка разрушительным слиянием (destructive merge) и загрузка созидательным слиянием (constructive merge). Основными критериями выбора техники загрузки, которая будет применяться на этапе Load в процессе ETL являются параметры самих данных, отправляемых в хранилище, а также – архитектура и цели существования хранилища. Рассмотрим техники загрузки подробнее.

Простая загрузка (Load) или процесс загрузки «по умолчанию». Пустые таблицы измерений и фактов заполняются данными, прошедшими предварительные процедуры трансформации. Как правило, это происходит после релиза хранилища данных. Если таблицы измерений и фактов, на момент загрузки, не являются пустыми, то все предыдущие данные стираются, после чего загружаются актуальные массивы данных.

Загрузка добавлением (Append) загружает в хранилище только те данные, дубликаты которых отсутствуют на момент загрузки массива в хранилище (дописываются недостающие данные). Данной технике сопутствует большое

количество спорных ситуаций, когда следует определить – действительно ли данные, уже имеющиеся в хранилище, являются дубликатами. Это создает дополнительную нагрузку на сервер хранилища данных (если конфликты разрешаются встроенными функциями или хранимыми процедурами) или на администратора хранилищ данных (если конфликты решаются вручную).

Загрузка разрушительным слиянием добавляет в хранилище новые данные, не имеющие дубликатов, а в случае обнаружения в хранилище дубликатов, заменяет их на новые значения.

Загрузка созидательным слиянием, в свою очередь, также добавляет в хранилище новые данные, не имеющие дубликатов, а в случае обнаружения дубликатов записывает дублирующиеся данные как новые значения в соответствующей таблице.

Схематично примеры исполнения разных техник загрузки показаны на рис. 26.



Рисунок 26. Техники загрузки данных в хранилище

Важно отметить, что загрузка данных в хранилище не заканчивает ETL процесс. После того, как данные оказались в хранилище, их следует еще раз проверить. Процедура проверки данных, загруженных в хранилище, называется **валидация**. Отдельно валидации подлежат данные типа «перечисление и текст», «числа и порядки» и «Даты и периоды». Особенности валидации данных в зависимости от их типа показаны на рис. 27-29.

Внутри поля	По отношению к другим полям	Совместимость форматов при передаче между системами
Не из списка разрешенных значений. Отсутствие обязательных значений. Несоответствие формату (например, «Все договора должны нумероваться «ДГВxxxx..»).	Не из списка разрешенных значений для связанного элемента. Отсутствие обязательных элементов для связанного элемента. Несоответствие формату для связанного элемента (например, «Для продукта «АИС» все договора должны нумероваться «АИСxxxx..»).	Символы допустимые в одном формате, недопустимы в другом. Разная кодировка. Отсутствие маппинга при добавлении/изменении меты данных. Устаревшие значения (не из списка разрешенных в целевой системе).

Рисунок 27. Валидация данных типа «Перечисление и текст»

Внутри поля	По отношению к другим полям	Совместимость форматов при передаче между системами
Не является числом. Не находится в границах разрешенного интервала значений. Пропущено порядковое значение (сбой в передаче пакета с данными).	Элементу «А» присвоен неправильный порядковый номер. Разницы за счет разных правил округления значений (например, в 1С и SAP может не «сойтись» рассчитанный НДС).	Переполнение (по ограничению). Потеря точности и знаков. Несовместимость форматов при конвертации в тип, отличный от числового.

Рисунок 28. Валидация данных типа «Числа и порядки»

Внутри поля	По отношению к другим полям	Совместимость форматов при передаче между системами
	День недели не соответствует дате. Сумма единиц времени не соответствует действительности, из-за разницы рабочие/не рабочие/праздничные/сокращенные дни.	Несовместимость формата даты при передаче текстом (например: ISO 8601 в UnixTime, или разные форматы в ISO 8601). Ошибка точки отсчета и точности при передаче числом (например: TimeStamp в DateTime).

Рисунок 29. Валидация данных типа «Даты и периоды»

Ввиду наличие дополнительного логического смысла, наибольшую сложность в процессе валидации представляет обработка символьных значений, процесс которой зачастую невозможно полностью автоматизировать. Валидация чисел и дат более упорядочена и, как правило, автоматизируется при помощи процедур и встроенных функций.

4.4. Вопросы для самостоятельного изучения по теме

1. Дайте краткое описание программному продукту Apache Airflow. Что такое DAG?
2. Перечислите известные вам функции агрегации данных. Дайте краткое описание их работы.

3. Какие еще, кроме Airflow и SSDT приложения для создания ETL пайплайнов вам известны? Перечислите и дайте краткую характеристику.

4.5. Тестовые задания для самопроверки

1. Процедуры ETL применяются на этапе...?

- А) сбора данных из источников;
- Б) перемещения данных из операционных баз в хранилище;
- В) формирования витрин данных;
- Г) формирования гиперкубов.

2. Что такое mapping в ETL процессе?

- А) разметка источников данных;
- Б) формирование схем таблиц в хранилище;
- В) формирование схем трансформации данных;
- Г) формирование схем витрин данных.

3. Пайплайн ETL это...

- А) последовательность преобразований потока данных;
- Б) процесс доставки данных в хранилище от сбора по источникам до верификации;
- В) процесс доставки данных в хранилище от сбора по источникам до загрузки в хранилище.

4. Какая из перечисленных задач трансформации данных не характерна для ETL процесса?

- А) конверсия;
- Б) агрегация;
- В) верификация;
- Г) насыщение;
- Д) все перечисленные задачи могут быть выполнены в ходе ETL процесса.

5. Насыщение данных, это...

- А) искусственное расширение массива входных данных;
- Б) изменение порядка и упрощение столбцов в таблице;
- В) сбор данных из доверенных источников;
- Г) повышение качества загруженных в хранилище данных.

6. При какой из перечисленных техник загрузки данных в хранилище предыдущий массив данных полностью заменяется?

- А) простая загрузка;
- Б) загрузка разрушительным слиянием;
- В) загрузка созидательным слиянием;
- Д) перезапись хранилища.

7. При какой из перечисленных техник загрузки данных в хранилище предыдущий массив данных дополняется всеми загружаемыми значениями?

- А) простая загрузка;
- Б) загрузка разрушительным слиянием;
- В) загрузка созидательным слиянием;
- Д) перезапись хранилища.

5. ТЕХНОЛОГИЧЕСКИЕ СТЕКИ ХРАНИЛИЩ ДАННЫХ ДЛЯ BIGDATA

5.1. Масштабируемое OLAP хранилища данных ClickHouse.

Распределенное MPP хранилище данных PostgreSQL GreenPlum

В условиях быстрого развития технологий сбора, обработки и хранения больших данных (BigData), активно развиваются и современные решения для построения соответствующих хранилищ данных. В настоящее время, количество предлагаемых на рынке ПО продуктов для хранения больших данных уже существенно превышает по количеству традиционные, реляционные программные решения. Под отдельные задачи хранилища применяются соответствующие типы хранилищ больших данных. Ниже, на рис. 30 приведены наиболее распространенные мировые и российские программные средства, позволяющие разворачивать хранилища больших данных.



Рисунок 30. Наиболее популярные в мире и в РФ программные продукты хранилищ больших данных

Краткое описание принципов функционирования и особенностей этих продуктов будет приведено в этой главе.

Хранилище «широких таблиц» Clickhouse для обработки аналитических OLAP запросов от компании Yandex. Структура в целом похожа на структуру классического OLAP хранилища данных, про которое было рассказано выше, с поправкой на специфику работы с большими данными.

Это хранилище считается «коробочным», то есть устанавливается из стандартного пакета-установщика, после чего конфигурируется под решение конкретных задач. Данное хранилище может быть развернуто в среде ОС Linux или MacOS, при этом используется стандартный набор драйверов загрузки данных: TCP, HTTP, MySQL, JDBC и ODBC драйверы. В качестве входных

источников данных могут быть использованы CSV файлы, локальные «плоские» файлы, результаты работы ETL процессов, прямая загрузка из реляционных БД, поддерживается работа брокеров сообщений для сервисных архитектур (в частности, интеграция с Apache Kafka). В качестве сервисов для аналитики и построения графической отчетности рекомендованы следующие программные продукты: DeepNote, Grafana, SuperSet, Tableau.

При масштабировании хранилища применяются технологии репликации и шардирования (про них будет рассказано в тематической лекции позже). Для обеспечения совместной работы всех узлов или шардов в кластере используется встроенное ПО ClickHouse Keeper или же стандартное решение для распределенных хранилищ – ZooKeeper.

Хранилища для больших данных, построенные на базе ПО ClickHouse развернуты не только в РФ (Яндекс.Метрика, ВКонтакте), но и используются крупнейшими мировыми компаниями (Uber, Spotify, DeutscheBank).

Основой языка скриптов в ClickHouse является модифицированный с учетом работы с большими данными и широкими таблицами диалект SQL. Вот некоторые специфические черты этого языка программирования:

- язык обладает дополнительными функциями для «приблизительных вычислений» внутри широких таблиц, что существенно сокращает время в случае возникновения необходимости выполнить аналитическую работу с данными относительно классических хранилищ данных;

- набор дополнительных функций для отдельных типов данных, например – возможность с помощью функции разложить адрес URL на составные части (имя протокола, имя домена, путь и физические имя файла и т.д.), когда это необходимо;

- в данном хранилище является нормальным явлением в качестве отдельных единиц хранения использовать массивы данных (array), которые будут храниться в одной ячейке, а также объединять в массивы даже несколько строк таблицы;

- внутри запросов свободно применяется map-reduce функция из фреймворка работы с большими данными;

- работа инструкции языка SQL JOIN заменена на подключение к запросам других таблиц, представляя их «внешними словарями».

Сочетание нескольких узлов с СУБД PostgreSQL, объединенные в единое целое с помощью программной надстройки GreenPlum – это очень популярное решение хранилищ больших данных OLP, построенных по принципу MPP

(Massive Parallel Processing), см. материалы лекции 3. Типовая архитектура технологического стека, построенного на базе ПО GreenPlum показана на рис. 31.

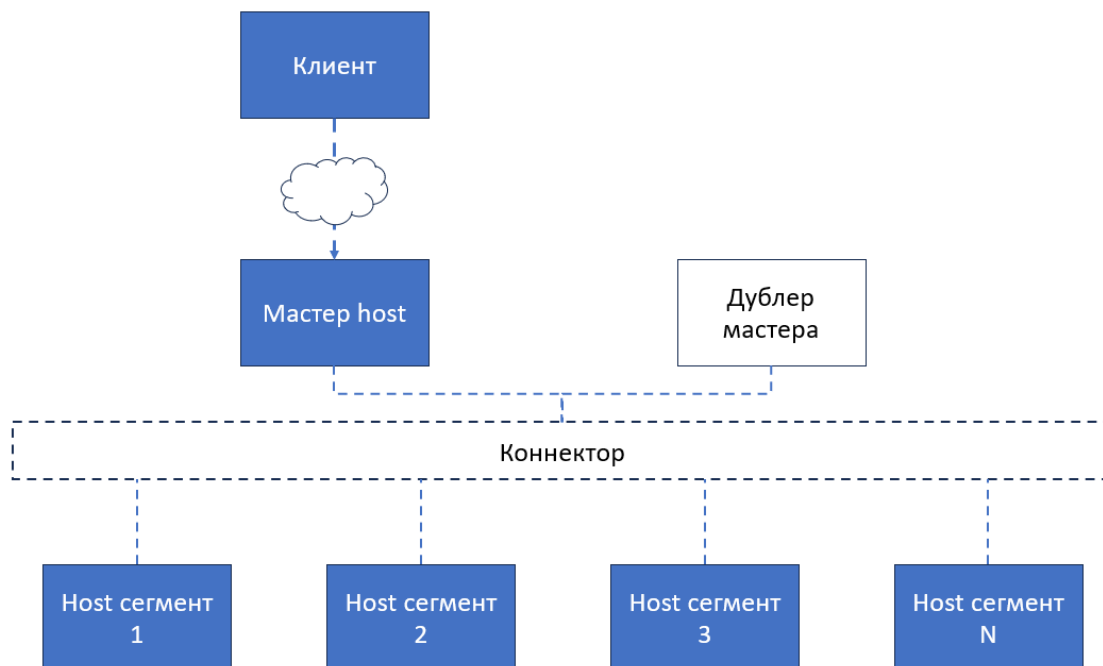


Рисунок 31. Схема типового технологического стека на базе ПО GreenPlum

Логика архитектуры хранилища данных, построенного на Host сегментах достаточно проста в понимании. Для создания эффективных распределенных баз данных устанавливается необходимое количество ядер серверов на базе ПО реляционной базы данных PostgreSQL, каждое отдельное ядро, в рамках архитектуры называется **Host сегмент**, и их может быть какое угодно количество. Такая архитектура практически не имеет ограничений по мощности, что является одним из главных показателей готовности хранилища к работе с большими данными. Далее, host сегменты, с помощью специального **коннектора** соединяется с **master host**, управляемым настроенным ПО GreenPlum, главной задачей которого является представление всех распределенных по host сегментам данных в виде единого, целого массива, а также, - предоставление инструментария работы с этим единым массивом. Учитывая важность бесперебойной работы мастер host для обеспечения доступа к данным, в архитектуре подразумевается наличие «горячего» (всегда готового к работе) **сервера-дублера**, который в случае возникновения проблем с мастер host, временно займет его место управления распределенными данными. Стратегия организации данных состоит из двух шагов: равномерное распределение по каждому доступному сегменту (распределение), разбиение данных каждого сегмента на подмножества (разделение).

Стратегия обработки запроса на мастер host состоит из четырех шагов: компиляция запроса, оптимизация запроса, составление оптимального плана запроса, распределение запроса по сегментам. «Точка выхода» результатов работы с сегментами – это также мастер host (массивы данных выгружаются через него).

Поскольку основной данной хранилища является традиционного ПО реляционных СУБД (PostgreSQL), возможности по развертыванию унаследованы у этой СУБД: RHEL, CentOS, Ubuntu, Oracle Linux (разные вариации дистрибутивов LINUX).

В качестве драйверов загрузки данных в хранилище могут быть использованы: PXF (Platform Extension Network), доступ с источниками SQL и HDFS; Greenplum-Kafka (интеграция с брокером обмена сообщениями Kafka), GreenPlum Stream Server (главная ETL утилита, «швейцарский нож ETL» в этой архитектуре хранилища), Spark Connector (интерфейс взаимодействия с системой Apache Spark) а также традиционные и проверенные JDBC и ODBC драйверы.

Хранилища для больших данных, построенные на базе технологического стека GreenPlum и узлов с PostgreSQL используются для решения задач крупнейших компаний как в РФ (Тинькофф, Сбербанк, Ростелеком), так и за рубежом (Badoo, Sony, Boeing, NASDAQ).

5.2. Типовой технологический стек хранилища больших данных на основе фреймворка Apache Hadoop

Apache Hadoop – это большой фреймворк для работы с большими данными, начиная с их сбора и заканчивая передачей запрошенных данных в аналитику. Основой работы фреймворка является хранение больших данных и возможность агрегации этих данных функциями MapReduce. Базовая часть фреймворка состоит из 8-ми ключевых программных продуктов.

1. Hadoop Distributed File System (HDFS) – распределенная файловая система для хранения больших данных.
2. Hadoop YARN – фреймворк MapReduce (первичные вычисления данных, основанные на агрегации) + средства управления ресурсами кластера хранилища данных (группа серверов-узлов, объединенная в единое целое) и менеджмента задач управления и администрирования хранилища данных.
3. Hive – утилита, преобразующая SQL-like запросы в серии map-reduce задач (поскольку HDFS работает только в парадигме map-reduce, такая конвертация необходима при первичной обработке данных в самой файловой

системе, без предварительного переноса их в витрины).

4. Pig – язык программирования высокого уровня для решения аналитических задач (один из многих доступных).

5. ZooKeeper – распределенное хранение конфигураций. Координация между серверами, клиентами и серверами.

6. Mahout – «движок» машинного обучения на больших данных.

7. Ambari – веб-приложение для управления и мониторинга системы, с дэшбордами.

8. Spark – вычислительный «движок» для данных, хранящихся в хранилище данных, построенным на базе фреймворка Hadoop.

Приведенное выше программное обеспечение подбирается, устанавливается и настраивается под конкретные функциональные требования к хранилищу данных. Пример сравнительно простого технологического стека хранилища данных, построенного на продуктах фреймворка Apache Hadoop с выходом на конечных пользователей в лице программ аналитического анализа показан на рис. 32.



Рисунок 32. Схема хранилища данных, построенного на продуктах фреймворка Hadoop

В данной схеме пользователь, в соответствующих приложениях создает в рамках своей операционной деятельности контент в виде данных. Из распределенных источников (пользователей может быть большое количество) данные собирает и «доносит» до файловой системы Hadoop программное средство Apache Flume. Далее, данные, попавшие в файловую систему Hadoop или обрабатываются при помощи функции map-reduce или через утилиту Apache Hive (конвертация SQL -> MapReduce и наоборот) передаются в подходящее

средство аналитики. В приведенном примере, это мощное программное средство RStudio.

Технологические стеки хранилищ данных на базе фреймворка Apache Hadoop развертываются в операционных системах группы LINUX (как правило, это RHEL, Debian, Ubuntu).

Поскольку система очень сложна в настройке (ввиду тех же особенностей работы с LINUX системами), как правило, используются готовые сборки или коробочные версии хранилищ в виде контейнеров или виртуальных машин (например, Cloudera Hortonworks или Oracle BigData и BigData Lite).

5.3. Вертикальные хранилища данных Vertica

Хранилища данных, построенные на базе программного продукта Vertica по своей сути, являются полной противоположностью, описанной в п.5.2. хранилищам на базе продукта GreenPlum.

Это колоночные (PostgreSQL - строковые), построенные на архитектуре MPP (Massive Parallel Processing) распределенные хранилища данных, оптимальные для решения аналитических задач. Сравнение колоночной и строковой структуры хранения приведено на рис. 33.

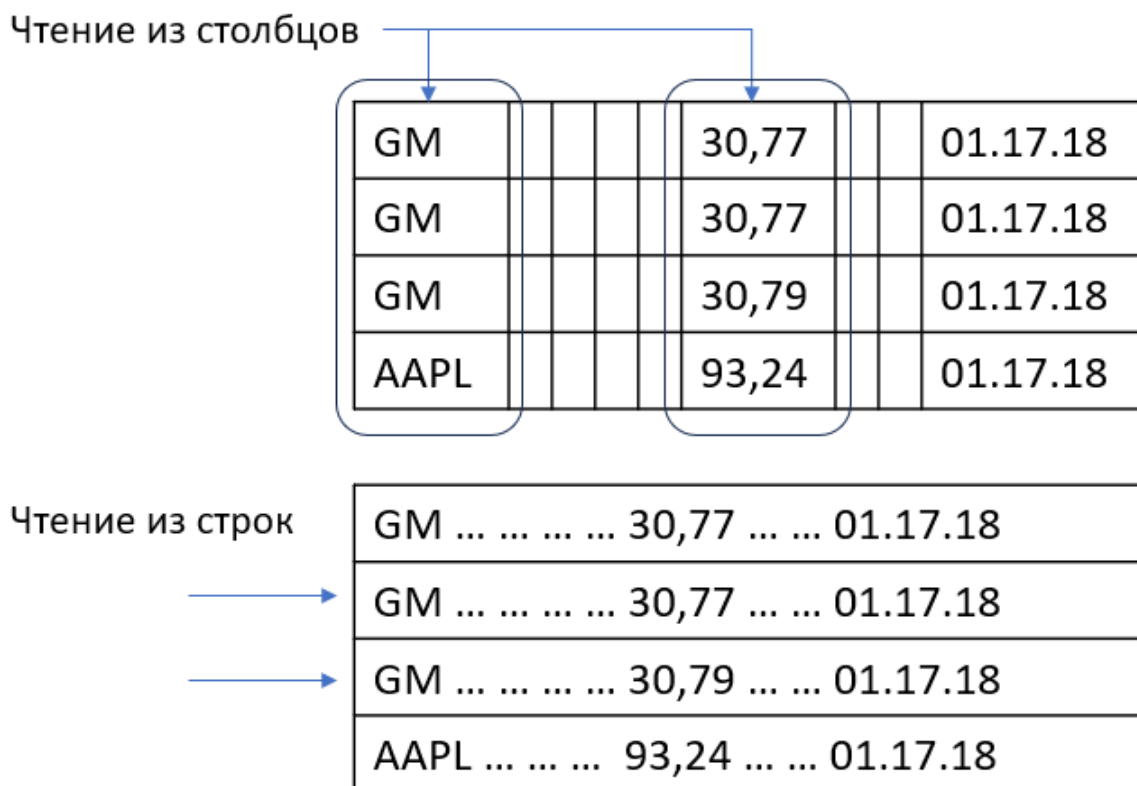


Рисунок 33. Сравнение колоночной и строковой моделей хранения данных

В отличие от GreenPlum, у таких хранилищ отсутствует единая точка

доступа, отказа и выдачи информации в лице мастер host, каждый узел (отдельный сервер) этой распределенной системы является независимым. Данные, попадая в хранилище равномерно балансируются по всем активным узлам распределенной системы. При этом – максимально просто расширить мощности при масштабировании хранилища данных, для этого буквально достаточно «воткнуть» новый сервер в «стойку» и подключить его через графический интерфейс системы управления. Бесперебойный доступ к данным обеспечивается на host дублером, как в случае GreenPlum, а одним или двумя серверами buddy projections (фактически, репликации), которые страхуют узлы хранилища на случай сбоев.

За счет того, что столбцы с однородными типами данных хранятся вместе, хранилище такого типа максимально быстро выдает данные в аналитику, а главной его проблемой является необходимость иногда перебалансировать распределение данных по узлам хранилища – это крайне ресурсоемкий и длительный процесс, без регулярного выполнения которого, хранилище через некоторое время будет испытывать серьезные проблемы с производительностью (одни узлы будут переполнены данными, другие при этом могут быть совершенно пустыми).

Хранилище данных, построенное на ПО Vertica, разворачивается в операционных системах RHEL, CentOS, Ubuntu, Oracle Linux, Debian.

В качестве драйверов загрузки данных в хранилище могут быть использованы: интеграция с брокером обмена сообщениями Kafka, собственные интерфейсы взаимодействия с продуктами из фреймворка Apache Hadoop, а также стандартный набор универсальных драйверов подключения к источникам данных JDBC, ODBC, ADO, OLEDB.

Для взаимодействия с хранилищем данных на основе Vertica используется стандартный набор инструкций языка SQL, расширенный набором аналитических функций (разные виды регрессионного, кластерный анализ и т.д.).

5.4. Вопросы для самостоятельного изучения по теме

1. Дайте определение и приведите схему работы программного средства ZooKeeper. Для чего оно применяется при работе с потоками данных?
2. Изучите и кратко опишите принцип работы программного приложения Apache Flume. На каком этапе работы с большими данными оно применяется? Каким образом конфигурируется?
3. Дайте краткую характеристику языку Pig Latin (Apache). Где и для чего он используется?

5.5. Тестовые задания для самопроверки

1. В чем заключается основная задача мастер-дублера в архитектуре хранилища на базе ПО GreenPlum?

- А) снятие нагрузки с основного сервера во время построения резервной копии хранилища данных;
- Б) страховка основного сервера в случае выхода его из строя;
- В) тестирование перспективных конфигураций;
- Г) управление узлами сервера.

2. Какова роль в фреймворке Hadoop программного средства YARN?

- А) машинное обучение больших данных;
- Б) сбор больших данных и перемещение в хранилище;
- В) менеджер ресурсов кластеров хранилища;
- Г) формирование и мониторинг ETL процедур;
- Д) мониторинг кластеров хранилища.

3. Какова роль в фреймворке Hadoop программного средства Ambari?

- А) машинное обучение больших данных;
- Б) сбор больших данных и перемещение в хранилище;
- В) менеджер ресурсов кластеров хранилища;
- Г) формирование и мониторинг ETL процедур;
- Д) мониторинг кластеров хранилища.

4. Какова роль в фреймворке Hadoop программного средства Ambari?

- А) машинное обучение больших данных;
- Б) сбор больших данных и перемещение в хранилище;
- В) менеджер ресурсов кластеров хранилища;
- Г) формирование и мониторинг ETL процедур;
- Д) мониторинг кластеров хранилища.

5. Какова роль в фреймворке Hadoop программного средства Flume?

- А) машинное обучение больших данных;
- Б) сбор больших данных и перемещение в хранилище;
- В) менеджер ресурсов кластеров хранилища;
- Г) формирование и мониторинг ETL процедур;
- Д) мониторинг кластеров хранилища.

6. Какой тип масштабирования характерен для хранилищ данных, построенных в ПО Vertica?

А) горизонтальное (увеличение единиц хранения);

Б) вертикальное (модернизация и актуализация имеющегося железа).

6. ДОКУМЕНТНАЯ МОДЕЛЬ ХРАНИЛИЩА ДАННЫХ MONGODB

6.1. Архитектура и программный инструментарий хранилища данных MongoDB. Базовые модели хранилищ MongoDB

Один из наиболее распространенных вариантов организации хранилища данных для Big Data по данным, по версии популярного ресурса <https://db-engines.com>, является построение его средствами программного продукта MongoDB, *документноориентированной СУБД*. Называется она так потому, что главной единицей хранения информации в такой СУБД является документ, подробнее об этом далее в лекции. Данный программный продукт относится к классу так называемых noSQL (not only SQL) баз данных, набравших существенную популярность в последние годы. Хранилища данных, развернутые в рамках данного продукта, легко горизонтально масштабируются (за счет модели шардирования, о которой будет рассказано в последующей лекции), а также работают с фреймворком MapReduce, который позволяет агрегировать числовые данные уже на стороне сервера, что очень удобно для работы аналитиков.

Помимо всех перечисленных достоинств, данная СУБД прекрасно показывает себя в работе с большими потоками данных, поскольку не требует строгой схемы при описании таблиц, в отличие от реляционной модели данных. Это позволяет хранилищу, построенному в MongoDB справляться с любыми потоками данных в режиме реального времени.

Долгое время, данная СУБД управлялась только встроенным консольным интерфейсом mongo shell, что создавало некоторые проблемы для неопытных пользователей, но затем появились удобные системы управления базой данных с графическим интерфейсом пользователя (MongoDB Compass), что кратно повысило эргономичность этой СУБД.

Сферы применения MongoDB для построения хранилища очевидно связаны с большими объемами генерации и передачи данных. Это: системы хранения и регистрации событий, электронная коммерция (сети ритейлеров), мобильные приложения социальных сетей, хранилища для поисковых роботов интернета, и т.д...

Основной программный инструментарий СУБД MongoDB состоит из следующего ряда сервисов:

- mongo – интерактивная оболочка, предоставляющая пользователю инструментарий управления сегментированием и шардированием сервера,

вставки/удаления/изменения имеющихся данных, выполнения запросов к хранилищу;

- mongostat – сервис ключевой статистики хранилища данных, визуализирует количество CRUD операций за отчетный период на сервере, ресурсопотребление по отдельным экземплярам сервера и т.д.;

- mongoimport/mongoexport – сервисы импорта и экспорта данных в хранилище, преимущественно «заточенные» на JSON-документы (модифицированный Java Script Object Notation – это нативный язык MongoDB);

- mongodump/mongorestore – инструменты администратора, для реализации ключевой функции резервного копирования и восстановления хранилища данных и его отдельных экземпляров.

Дадим основные определения элементам хранилища данных MongoDB.

Хранилище данных – упорядоченный набор файлов, расположенных в файловой системе сервера, который содержит внутри себя коллекции.

Коллекция – группа документов MongoDB, объединенных логическим путем (документы имеют связанное друг с другом, или аналогичное значение). Коллекции не являются таблицами в широком понимании реляционной модели хранения, поскольку не подчиняются реляционным правилам и не применяют схемы с пользовательскими ограничениями, порядком столбцов и типами данных. Каждый документ в коллекции может иметь свой набор столбцов.

Документ – это набор пар ключ-значение с динамической схемой. По умолчанию, ключ представляет собой объект ObjectID с дополнительной системой кодирования. Значения для разных документов могут отличаться друг от друга по набору столбцов, что усложняет запросы к данным, но существенно упрощает их масштабирование.

На рис. 34 показан пример коллекции «Сотрудники» хранилища MongoDB, состоящей из двух документов.

```

_id: 1
id_staffs: "s1"
sff_telno: "8-906-794-89-63"
sff_fullname: "Семенова"
sff_passp_data: "ОУФМС России по г. Новочеркасск 2018-03-30 4163 195322"
sff_experience: 5
sff_address: "Россия г. Сергиев Посад Пушкина ул. д. 19 кв.62"
pot_name: "Кондитер"
idt_name: "Какао-порошок"
rom_name: "Цех упаковки"

```

```

_id: 2
id_staffs: "s2"
sff_telno: "8-916-789-23-45"
sff_fullname: "Петров"
sff_passp_data: "Отделением УФМС России по г. Нефтекамск 2017-06-22 4287 658064"
sff_experience: 2
sff_address: "Россия г. Рязань Дзержинского ул. д. 23 кв.8"
pot_name: "Инженер-технолог"
idt_name: "Сахар"
rom_name: "Цех создания шоколада"

```

Рисунок 34. Коллекция Сотрудники, состоящая из двух документов

В данном примере используется искусственно созданный ключ `_id`, который говорит о том, что используется псевдореляционная или нормализованная модель хранилища MongoDB. В коллекции находятся два документа с номером 1 и 2, это кондитер и инженер-технолог некоторого производственного предприятия. По своей структуре на выходе, коллекция представляет собой стандартный документ нотации JSON, что существенно облегчает обмен этими данными через API приложений, использование их в веб-приложениях и сервисах, а также в современных аналитических инструментах.

Как было отмечено выше, хранилище MongoDB может быть построено в рамках одной из двух логических моделей (схем данных): модель данных со встроенными документами или нормализованная (псевдореляционная модель данных). Посмотрим их схему и разберем их особенности.

Модель хранилища со встраиванием (также называется «обогащенный документ mongodb») представляет собой набор коллекций, внутри которых документы организовываются по принципу «матрешки», вкладываются друг в друга. Для наглядности, схематично представим один из документов, изображённых на рис. 34 в виде документа с группой вложенных документов (рис. 35).

```
{
  _id: 1
  sff_fullname: "Семенова"
  contact: {
    sff_telno: "8-906-749-89-63"
  }
  profinfo: {
    sff_experience: 2
    pot_name: "Инженер-технолог"
  }
}
```

Рисунок 35. Образец документа с вложениями

Основным документом в схеме является документ с номером сотрудника «1». Внутри него находятся данные о имени сотрудника. Информация о номере телефона сотрудника находится во вложенном документе «contact». Таким образом, внутри одного документа, контактные данные сотрудников (а там еще может быть адрес, номер рабочего телефона и т.д., в виде массива), могут быть отделены от информации другого характера. Также, во вложенном документе «profinfo» находится информация о должности сотрудника и его трудовом опыте.

Данная схема является основной для хранилищ типа MongoDB и показывает максимальную эффективность в работе.

Часто бывает, что компания лишь по ходу своей деятельности сталкивается с необходимостью изменить подход к формированию хранилищ данных. Например, долгое время компания обходилась традиционным реляционным хранилищем данных, и лишь последовательное, в течение нескольких лет, расширение бизнеса, вкупе с существенным увеличением потока обрабатываемых данных, заставляет перейти на технологии по работе с большими данными. В этом случае, переход от legacy – хранилища данных, построенного по реляционным принципам, к документоориентированному хранилищу данных будет значительно проще, если в хранилище применить нормализованную (псевдореляционную) модель MongoDB. В этом случае, основу будет представлять несколько документов, объединенных между собой ссылками друг на друга (не путать с реляционными связями). Образец документа из нормализованной модели показан на рис. 36.

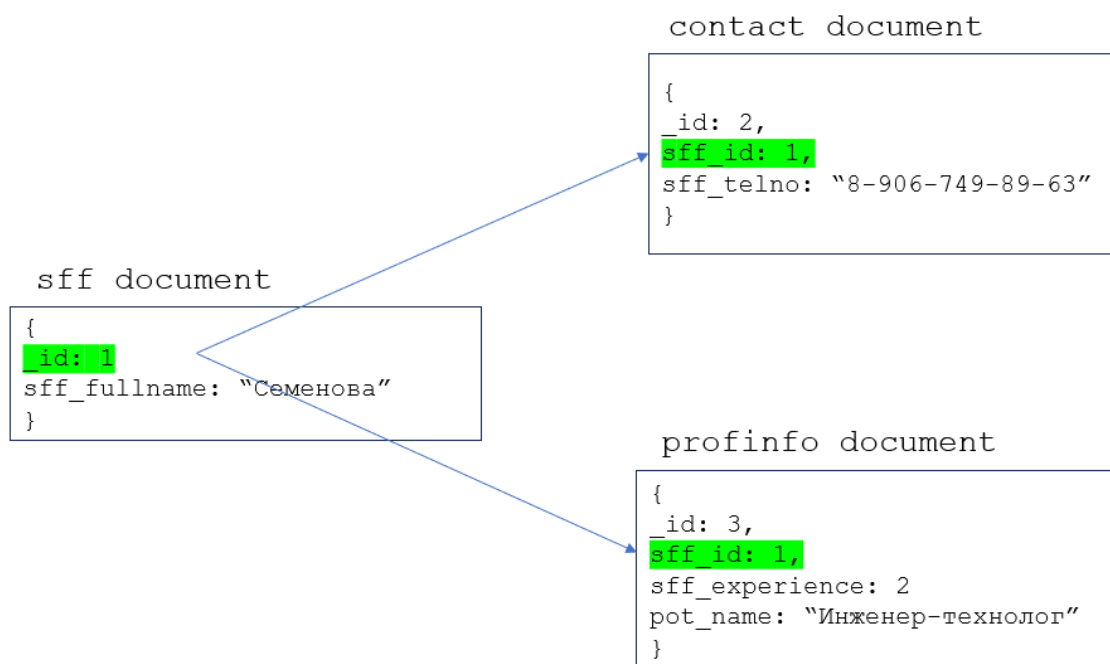


Рисунок 36. Образец документа нормализованной модели

В нормализованной модели в коллекции значительно больше документов, а их структура внешне напоминает реляционные сущности и связи. Но на этом сходства моделей заканчиваются. Связи в модели документов не позволяют полноценно формировать массивы данных на основании нескольких таблиц, как это происходит, например с инструкциями JOIN в реляционной модели хранения.

6.2. Архитектурные паттерны элементов хранилища MongoDB

При создании хранилищ данных на базе СУБД MongoDB, также как и в случае с традиционными хранилищами данных, рассмотренными выше, при решении некоторых задач, связанных с оптимизацией структуры хранилища, возможно применение готовых моделей, хорошо зарекомендовавших себя в эксплуатации. Эти модели могут быть использованы для организации всего пространства хранилища или его части и называются **архитектурными шаблонами (паттернами)**.

Ниже будут описаны и показаны схематически варианты архитектурных шаблонов, применяемых для решения типовых задач при проектировании хранилищ данных СУБД MongoDB.

Модель вложенных «один к одному» документов. Это традиционная модель с вложенными документами, описанная выше (см. рис. 35). Ее целесообразно применять при необходимости хорошо структурировать данные (например, перед выгрузкой в документы JSON), и когда в созданных

коллекциях находятся документы с большим количеством столбцов внутри. Для перехода к этой модели, коллекции исследуются на предмет наличия в них данных, которые можно «каталогизировать» во вложенном виде. Вся однородная информация отправляется в соответствующий ей вложенный документ. Пример перехода от стандарта хранения к модели вложенных «один к одному» документов показан на рис. 37.

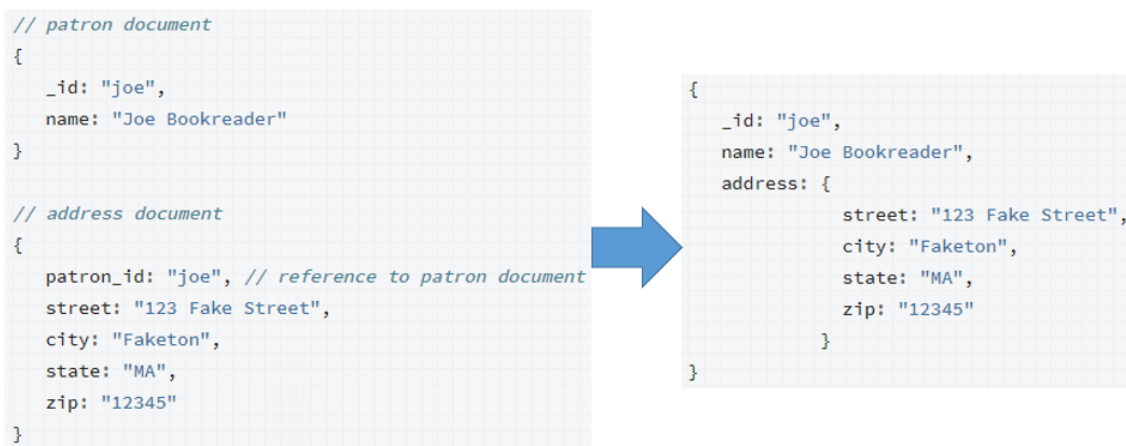


Рисунок 37. Переход к модели вложенных «один к одному» документов

На рисунке представлен один документ из коллекции «Покупатели», в которой хранится информация о покупателях книг в магазине. Приведены имя покупателя и дополнительная информация, связанная с его местом проживания. Очевидно, что данные о улице, городе, штате, и почтовом коде – это однородные данные, которые можно поместить во вложенный документ, например, с названием «Адрес». Обратим ваше внимание на то, что показанный пример не является настоящей моделью хранилища данных и используется только для демонстрации. Очевидно, с такими чувствительными данными, как место проживания покупателя надо поступать существенно «аккуратнее», чем в приведенной модели (речь про шифрование, перенос на отдельный защищенный сервер хранилища и т.д.).

Модель вложенных «один ко многим» документов. Эта модель является производной от модели вложения «один к одному» и применяется в случае, когда одни и те же вложенные документы применяются в большом количестве документов более высокого порядка. В случае хранения больших данных, особое внимание следует обращать на использование повторяющихся данных. Большой процент дублирования данных в хранилище существенно повышает его объем, требует большие мощности, и соответственно, - стоит дороже. Пример перехода

от модели с вложением «один к одному» к модели с вложением «один ко многим» показан на рис. 38.



Рисунок 38. Переход от модели вложения «один к одному» к модели «один ко многим»

На рисунке представлен один документ из коллекции «Книги», в которой хранится информация о продаваемых в интернет-магазине образовательных книгах. Цветом выделен вложенный документ, неоднократно повторяющийся во многих документах – данные (название, место расположения офиса) об издателе. Крупных издателей не так много, и очевидно, что эта информация будет часто повторяться, поэтому целесообразно, чтобы избавиться от многократного дублирования, перейти к модели «один ко многим».

Для этого необходимо создать отдельный документ для каждого издателя (например, в коллекции «Издатели») и внутри этого документа, в массиве значений Книги (books, представляет собой последовательность значений, разделенную запятыми и помещенную в скобки []) указать порядковые номера (ключи) всех книг, выпущенных данным издательством и продаваемых в магазине. В дальнейшем можно будет убедиться, что массивы данных в хранилищах данных, построенных на PostgreSQL СУБД, играют более значимую роль и встречаются значительно чаще, нежели массивы в классических хранилищах.

Модели дерева «снизу вверх» и «сверху вниз». В структуре данных хранилищ часто встречаются естественные иерархии – случаи, когда вместе хранятся данные разной грануляции. Термин грануляция уже применялся

раньше в контексте описания традиционных хранилищ данных, как инструмент обеспечения удобства работы конечных приложений аналитики данных. Например, благодаря грануляции, в зависимости от выбранного атрибута измерения «Время» (месяц, год, день, неделя и т.д.), исследователь может сразу получить срез данных выбранного масштаба. Такие грануляции характерны и для документноориентированных хранилищ данных. В этом случае, для эффективной работы с такими иерархиями, применяют архитектурный шаблон дерева. При движении по дереву иерархии «снизу вверх» описываются документы-потомки и затем дается указание на родителя, находящегося «выше» по дереву (более высокого порядка). При движении по дереву иерархии «сверху вниз» напротив, для всех документов-родителей последовательно прописываются ссылки на документы-потомки. Для наглядности, изучим эту структуру на примере. Модель для изучения (иерархия) представлена на рис. 39.

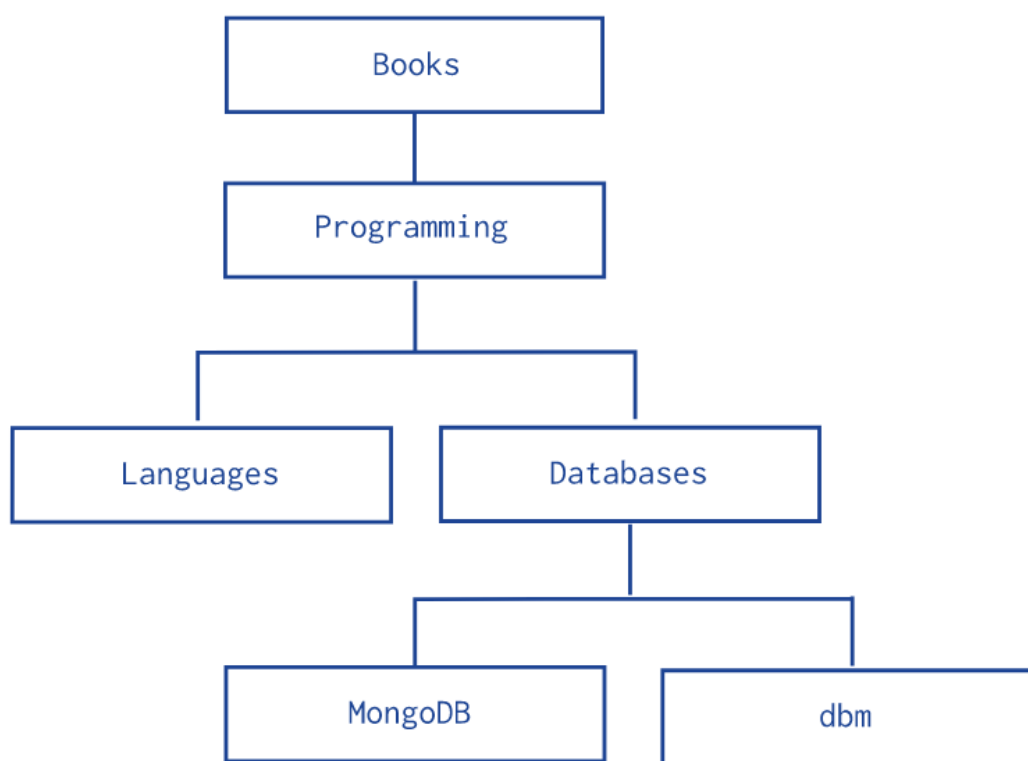


Рисунок 39. Образец иерархии документов

Тематика «книги» в приведенной иерархии постепенно декомпозируется от верхнего уровня – непосредственно книги, до нижнего – книги с названиями и своим ISBN (например, следующее: Книги-Программирование-Базы данных-MongoDB).

На рис. 40 показан скрипт создания в рамках коллекции «Категории» документов книгами, организованных, как Дерево «снизу вверх».

```

db.categories.insert( { _id: "MongoDB", parent: "Databases" } )
db.categories.insert( { _id: "dbm", parent: "Databases" } )
db.categories.insert( { _id: "Databases", parent: "Programming" } )
db.categories.insert( { _id: "Languages", parent: "Programming" } )
db.categories.insert( { _id: "Programming", parent: "Books" } )
db.categories.insert( { _id: "Books", parent: null } )

```

Рисунок 40. Модель документов Дерево «снизу вверх»

Начиная с книги по MongoDB, находящейся на низшей ступени иерархии, последовательно, с помощью свойства parent (родитель), в котором указывается более высокий уровень иерархии, создается вся структура дерева. Обратите внимание, что во всех случаях, один потомок соответствует своему одному родителю. В последней строчке, где описывается верхний уровень иерархии, для описания родителя документа Books применяется неопределённое значение null.

На рис. 41 показан скрипт создания в рамках коллекции «Категории» документов книгами, организованных, как Дерево «сверху вниз».

```

db.categories.insert( { _id: "MongoDB", children: [] } )
db.categories.insert( { _id: "dbm", children: [] } )
db.categories.insert( { _id: "Databases", children: [ "MongoDB", "dbm" ] } )
db.categories.insert( { _id: "Languages", children: [] } )
db.categories.insert( { _id: "Programming", children: [ "Databases", "Languages" ] } )
db.categories.insert( { _id: "Books", children: [ "Programming" ] } )

```

Рисунок 41. Модель документов Дерево «сверху вниз»

При движении от документов-родителей к потомкам сразу можно отметить, что, при указании ссылки (например, с помощью свойства документа children, потомок) используются массивы. Очевидно, это связано с тем, что у родителя может быть несколько потомков одного уровня иерархии. В остальном, эта схема похожа по своей логике на описанную выше модель дерева «снизу вверх».

Модель поиска по тэгам. Тэги (ярлыки) – это некоторое ключевое слово, которое может ассоциироваться с определенным объектом информации. Тэги позволяют пользователям находить в сети Интернет интересующий их контент. Так, например, тэг «фото забавных котиков», введенный в поисковой строке, позволит вам найти немало веселых фотографий замечательных домашних (и не очень) животных. Учитывая ориентированность поSQL хранилищ данных на работу с большими данным, работа с тэгами – это весьма распространенная задача для построения в рамках схемы хранилища. При использовании моделей поиска по тэгам, все тэги каталогизируются с помощью специально создаваемого

для этого индекса, а затем, для каждого документа, имеющего набор тэгов, эти тэги помещаются в массив. При поиске – достаточно указать один или несколько тэгов, которые интересуют пользователя. Пример модели показан на рис. 42.



Рисунок 42. Образец модели поиска по тэгам

При добавлении в коллекцию Книги, для каждого документа создается массив с присущими ему тэгами (для книги «Моби-Дик» это свойство topics с массивом: Охота на китов, Аллегория, Месть, Американцы, Роман, Морская тематика, Путешествия, Кейп-Код). С помощью инструкции createIndex, для массива topics создается специальный индекс. Далее, при поиске (findOne/findMany), достаточно указать необходимые для поиска тэги.

Модель «дата-время». Данная модель применяется в случае, если основной массив данных каких-то документов в коллекции состоит преимущественно из количественных показателей, привязанных к дате и времени. Например, при постоянном получении данных от каких-нибудь датчиков температуры в сложном производстве. В этом случае, множество маленьких документов с результатами измерений приводятся к одному «широкому» документу. Пример этой модели показан на рис. 43.



Рисунок 43. Образец модели «дата-время»

Все данные из документов коллекции «Температуры», содержащие данные о последовательном измерении, полученном на счетчике с номером 12345, перемещаются в один документ с номером 1, в массив «Измерения». При этом добавляется информация о дате-времени начала измерений, и, если это необходимо, - дате-времени окончания измерений.

6.3. Инструменты визуализации модели данных MongoDB

В настоящее время нет общепринятых нотаций визуализации логической модели данных MongoDB. В первое очередь это связано с тем, что, поскольку документоориентированное хранилище данных поддерживает работу с несогласованными данными, группы проекта BigData хранилищ данных, как правило, пропускают этап логического проектирования.

Однако, учитывая стремление привести любую noSQL базу данных к конечной согласованности данных (приведение коллекций к однородной схеме для всех экземпляров), вместе с большим количеством коллекций в типовой модели хранилища данных наводит на мысль об удобстве использования в ходе проектирования промежуточных, логических моделей хранилища, по аналогии с классическими типами баз данных и хранилищ.

В среде разработчиков большой популярностью в качестве инструмента моделирования пользуются программные продукты, позволяющие изобразить структуру комплексных JSON-документов. Одним из них является мобильное приложение JSON Designer. Образец визуализации модели вложенных JSON-документов (аналогична модели вложенных документов MongoDB, см. рис. 35) показан на рис. 44.

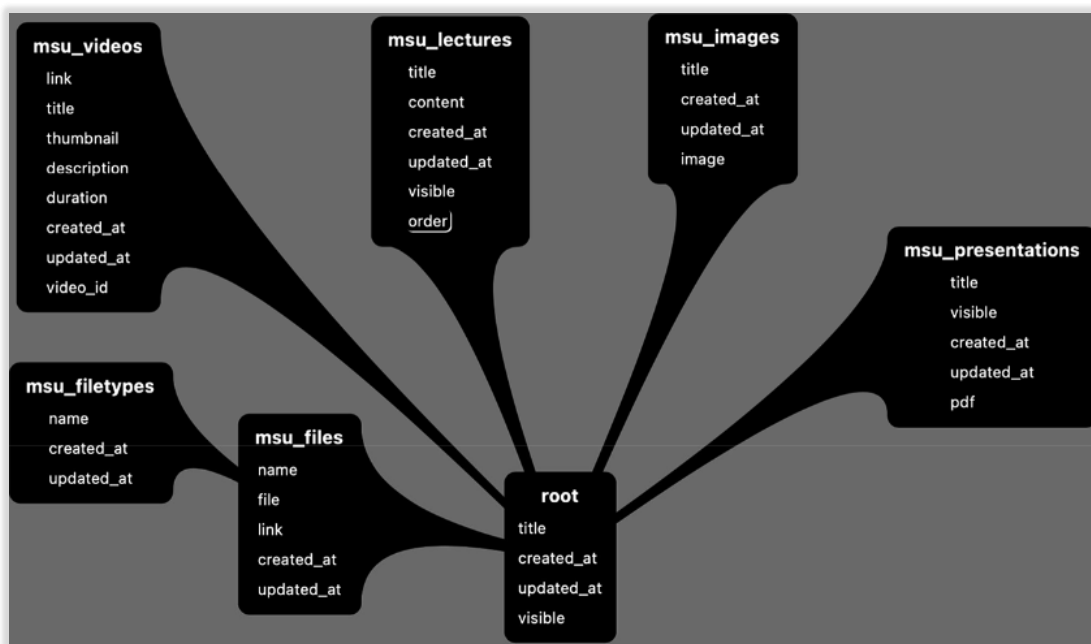


Рисунок 44. Образец модели вложенных JSON-документов

На рисунке показана структура документов для организации работы веб-приложения для хранения и демонстрации графических и видеоматериалов лекций для студентов (<https://msuniversity.ru>). Объекты, похожие на капли нефти – это документы верхнего уровня, в некоторых из них допустимы вложения. Вложенные документы показаны особым образом, такой документ `order`, на высоком уровне абстракции, можно увидеть в документе верхнего уровня `msu_lectures`. Нотация позволяет также подробно декомпозировать вложенные документы. При этом, всегда, объектом верхнего уровня является не хранилище данных, а документ `root`.

В случаях, когда речь идет не о связанных документах, а о полноценном хранилище, в частности построенном в рамках псевдореляционной модели данных (см. рис. 36), нотации с JSON-документами недостаточно. Для демонстрации полноценной документной прибегнем к способу визуального сравнения. Ниже, на рис. 45 и рис. 46 будут последовательно приведены стандартная реляционная модель хранения данных (`legacy`) и документная модель данных, построенная на ее основе, при переходе от `legacy` к документоориентированному хранилищу данных, построенному в рамках псевдореляционной модели.

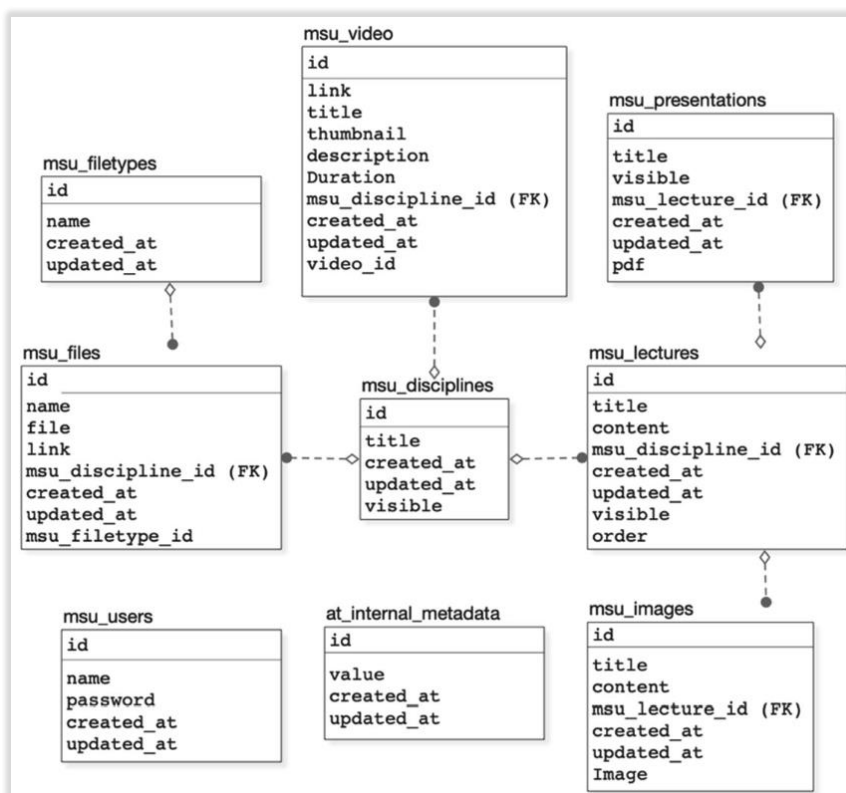


Рисунок 45. Реляционная модель данных веб-приложения для проведения лекционных занятий

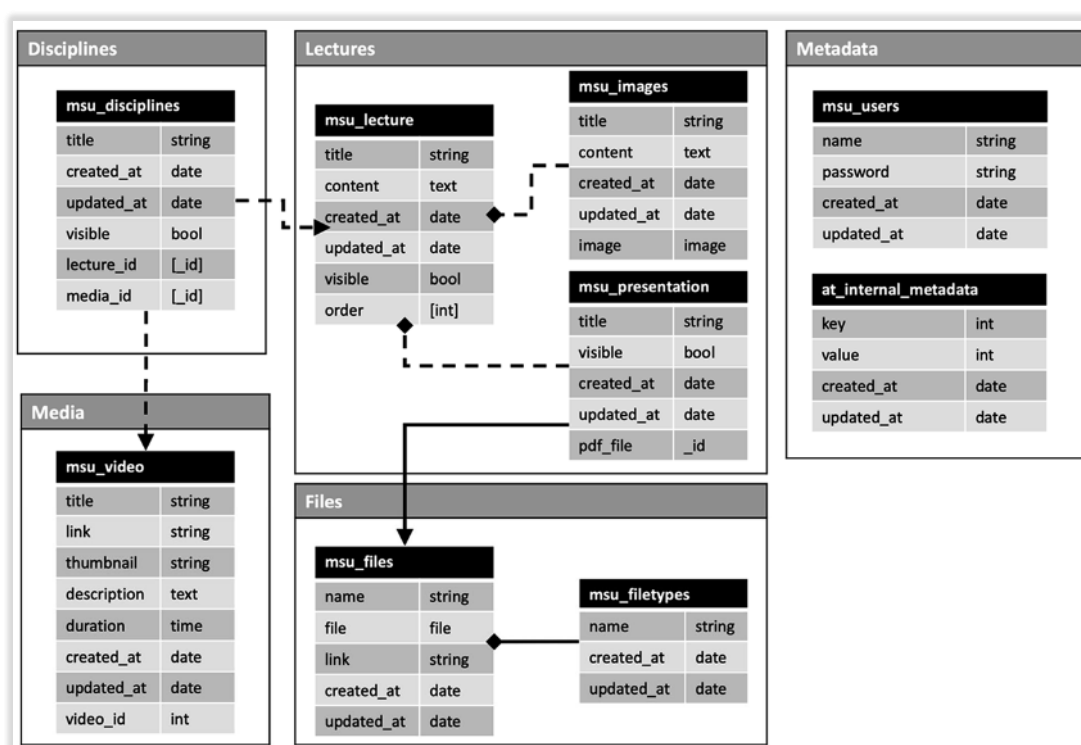


Рисунок 46. Документная модель данных веб-приложения для проведения лекционных занятий

Документы в коллекциях на рис. 46 представлены в виде контейнеров.

Контейнер может содержать как один, так и несколько документов (в случае, если они вкладываются друг в друга). Каждый документ в коллекции имеет название и стандартный набор типов данных для атрибутов. Более подробно, работа с документной моделью данных будет рассмотрена в рамках практических занятий по дисциплинам, связанным с проектированием хранилищ данных.

6.4. Вопросы для самостоятельного изучения по теме

1. Дайте краткое объяснение смысла применения ключа ObjectID для документов MongoDB.
2. Приведите определение и краткое описание JSON и BSON документов.
3. Приведите варианты тэгов для модели поиска по тэгам фрагмента хранилища данных с данными об автомобильных комплектующих.

6.5. Тестовые задания для самопроверки

1. Какой из перечисленных программных продуктов не находится в актуальной первой десятке рейтинга db-engines:
А) Neo4j;
Б) Elasticsearch;
В) Snowflake;
Г) IBM DB2.
2. Какой из перечисленных классов программных продуктов баз данных не находится в актуальной первой десятке рейтинга db-engines:
А) Search engine;
Б) Document;
В) Key-value;
Г) Wide column.
3. Единицей хранения хранилища данных MongoDB является:
А) Слот;
Б) Поле;
В) Документ;
Г) Экземпляр.

4. Группа объектов хранения в хранилище данных MongoDB называется:

- А) Таблица;
- Б) Сущность;
- В) Коллекция;
- Г) Колонка.

5. Сервис MongoDB, отвечающий за резервное копирование данных в хранилище называется:

- А) mongos;
- Б) mongobackup;
- В) mongodump;
- Г) mongorepl.

7. Сервис MongoDB, отвечающий за управление хранилищем называется:

- А) mongo;
- Б) mongocontrol;
- В) mongosh;
- Г) mongos.

8. Хранилище данных MongoDB «дерево» может быть реализовано в следующих вариациях:

- А) «снизу-вверх» и «сверху-вниз»;
- Б) «слева-направо» и «справа-налево»;
- В) «по убыванию» и «по возрастанию»;
- Г) все варианты являются правильными.

9. При реализации модели «поиск по тегам» обязательно использование следующего элемента хранилища данных:

- А) хранимая процедура;
- Б) триггер;
- В) последовательность;
- Г) индекс.

7. ШАРДИРОВАННЫЕ КЛАСТЕРЫ ХРАНИЛИЩА ДАННЫХ MONGODB

7.1. Элементарная схема хранилища данных MongoDB. Проблема масштабирования хранилища для больших данных

Современные потоки данных для крупных организаций оцениваются терабайтами и петабайтами информации, поступающей из большого количества распределенных источников, что, помимо тривиальных задач, связанных с процессами ETL, ставит перед проектировщиками еще и задачи, связанные с масштабированием хранилищ данных BigData. Иными словами, «железо» и программное обеспечение хранилищ BigData должны обладать потенциалом масштабирования (увеличения доступных мощностей) в максимально сжатые сроки.

Выделяют вертикальное и горизонтальное масштабирование,

Под **вертикальным масштабированием** мощностей сервера хранилища данных понимается достижение дополнительных возможностей при помощи улучшения «железа» сервера. Оперативная и физическая память нового поколения, более мощные многоядерные процессоры и даже использование оперативной памяти видеокарты при вычислениях увеличивают возможности рабочего сервера хранилища данных. Однако, рано или поздно, такой традиционный способ масштабирования мощностей «упрется в потолок» текущих возможностей компьютерной техники и потенциал роста будет исчерпан до следующей продвинутой итерации компьютерных комплектующих. К тому же, каждый очередной переход на использование все более продвинутых технологий будет обходиться все дороже, и цена прироста единицы мощности также будет расти.

Также, проблемой является то, что большие данные в некоторых предметных областях могут иметь не только большой объем хранения, но и в реальном времени приходить из источников в большом объеме, что при заполнении текущих серверов приведет к потере информации в течение вертикального масштабирования (даже добавление дополнительной единицы хранения к имеющемуся серверу занимает определенное время).

Альтернативой вертикальному масштабированию является **горизонтальное масштабирование**. В этом случае, увеличение вычислительной мощности хранилища достигается путем увеличения количества единиц хранения (узлов-серверов в кластере) и последующего равномерного (насколько это возможно) распределения данных по узлам. В

современном ПО хранилищ данных BigData, для добавления нового узла обычно достаточно подключить его к сети, установить серверное ПО и указать единой среде управления хранилищем данных логические параметры нового узла. Проблемой такого варианта наращивания мощностей является непосредственно организация распределенного хранилища данных, находящейся на большом количестве узлов-серверов, а также администрирование, мониторинг и управление ее работой. Именно решением этой проблемы и занимаются создатели современного программного обеспечения для серверов хранения BigData.

В MongoDB проблема распределенного хранилища данных с удобным и быстрым горизонтальным масштабированием стандартно решается путем создания структуры кластеров, состоящих из узлов. Каждый из узлов содержит свой фрагмент распределенного хранилища данных, а объем информации, хранящийся на каждом узле, автоматически балансируется в режиме реального времени (чтобы не было переполнения одних узлов и не пустовали другие).

7.2. Стандарт шардирования хранилища, варианты схем шардирования MongoDB

Типовой моделью распределенного хранилища данных является шардированная схема хранилища MongoDB с балансировщиком. Рассмотрим на рис. 47 типовую схему этой модели.

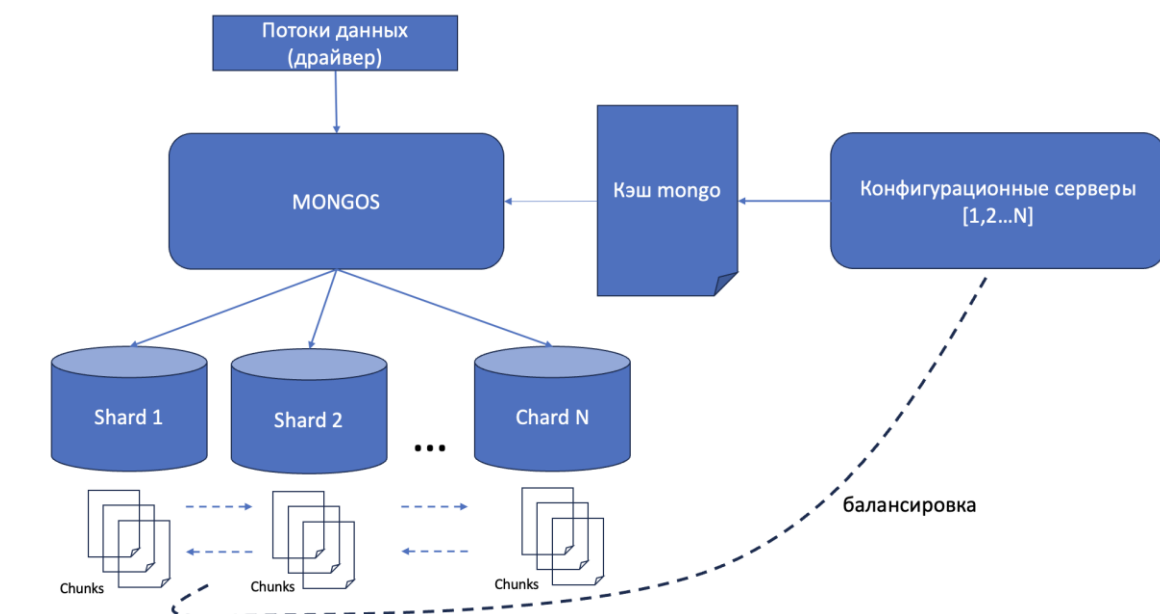


Рисунок 47. Типовая шардированная схема хранилища данных MongoDB с балансировщиком

Определим ключевые понятия схем шардирования MongoDB.

Кластер – единица хранения высокого уровня. Включает в себя процесс управления mongos, конфигурационный сервер и определенное требованиями количество экземпляров шардов.

Конфигурационный сервер MongoDB содержит в себе метаданные хранилища (репозиторий метаданных), а также все физические адреса отдельных чанков (chunk) данных на всех шардах кластера. Отдельной важной задачей конфигурационного сервера является равномерная балансировка входящих данных, учитывая параметры быстродействия и текущей заполненности шардов в автоматическом режиме, а также по команде администратора с уже сохраненными данными.

Процесс mongos (роутер) – инструмент передачи данных от пользовательских приложений и баз данных к шардам.

Шарды – фрагменты единого распределенного набора данных хранилища.

Чанк – единица хранения данных в шардированной схеме. Объем чанка – от 32 до 64 МБ.

Применение шардирования обусловлено не только необходимостью упрощения горизонтального масштабирования, но и удобством самой конструкции шардов. Благодаря относительной независимости шардов (каждый располагается на узле виртуального кластера), упрощается решение задачи построения геораспределенных систем (территориально отдаленных друг от друга кластеров). Выход из строя одного из элементов хранилища данных (один или несколько шардов) не остановит его работу, а лишь временно сократит доступный к использованию объем данных (и то – не во всех шардированных моделях). Шарды также повышают эффективность выполнения множественных запросов к хранилищу – в рамках одного шарда запросы могут выполняться параллельно.

Основной проблемой в применении принципа разделения хранилища данных на многочисленные шарды является сбор данных этих всех фрагментов в единое распределенное хранилище в сервисе mongos. Для обеспечения скорости работы сервиса был разработан элемент **ключ шарда**. Это поле или несколько полей, которые обязательно присутствуют в каждом документе коллекции, создаваемой в пространстве шарда. В момент, когда коллекция отправляется или создается в шарде, для нее определяется ключ шарда. Единожды определенный для коллекции ключ шарда изменить уже нельзя. При определении полей для ключа шарда преимущество отдается полям с наиболее случайными значениями (по многим принципам, определение ключа шарда

очень похоже на определение первичного ключа реляционной таблицы).

Следующая задача, которая стоит перед шардом – это представление единицы данных в удобном для запроса виде. Напомним, что применительно к хранилищам, построенным на базе PostgreSQL модели хранения, речь идет о больших массивах данных. Одна коллекция может содержать в себе документы на сотни мегабайт, а то и гигабайт данных, что может создавать серьезные проблемы с метриками времени и быстродействия процессов запроса к этим данным. Для решения задачи оптимизации запросов к данным, документы в шарде дробятся на «пеньки» или **чанки**. При создании документа, ему призывается первый чанк. При достижении этим чанком объема в **64 мегабайта**, примерно по среднему значению ключа шарда, этот документ делится на два чанка по 32 мегабайта. При достижении их объема в 64 мегабайта, они, в свою очередь делятся еще на два, и так далее. Таким образом, весь массив документа в частности и коллекции в целом делится на сравнительно компактные и удобные для запросов кусочки, обработать которые для современных серверов не составляет труда.

С помощью вышеупомянутых чанков происходит также крайне важный процесс балансировки данных. Под **процессом балансировки** данных понимают перманентный (постоянный) процесс, мониторящий распределение чанков по шардам, определение минимально и максимально заполненных шардов и перераспределение между ними чанков. Процесс переноса чанка с одного шарда на другой в рамках балансировки называется **миграцией** и происходит полностью в автоматическом режиме. Схематически он показан на рис. 48.

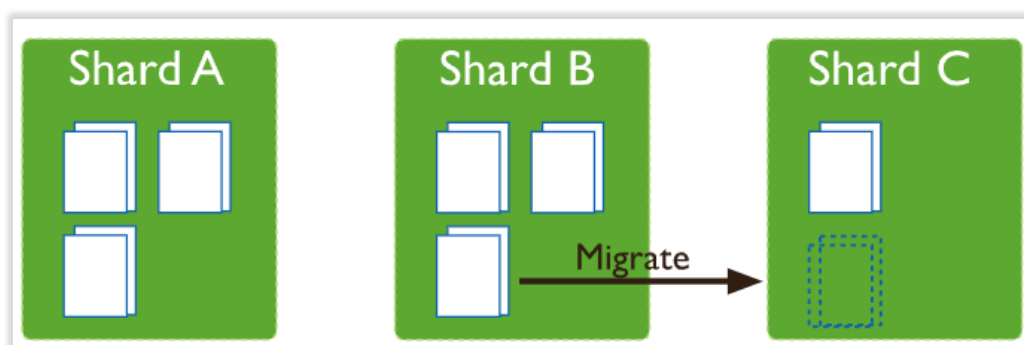


Рисунок 48. Миграция чанков с данными при балансировке

С целью обеспечения непрерывного доступа к ценной информации, находящейся в хранилище (защита от сбоев в кластерах). Практикуется использование модели шардированной схемы с балансировщиком и репликацией (см. рис. 49).

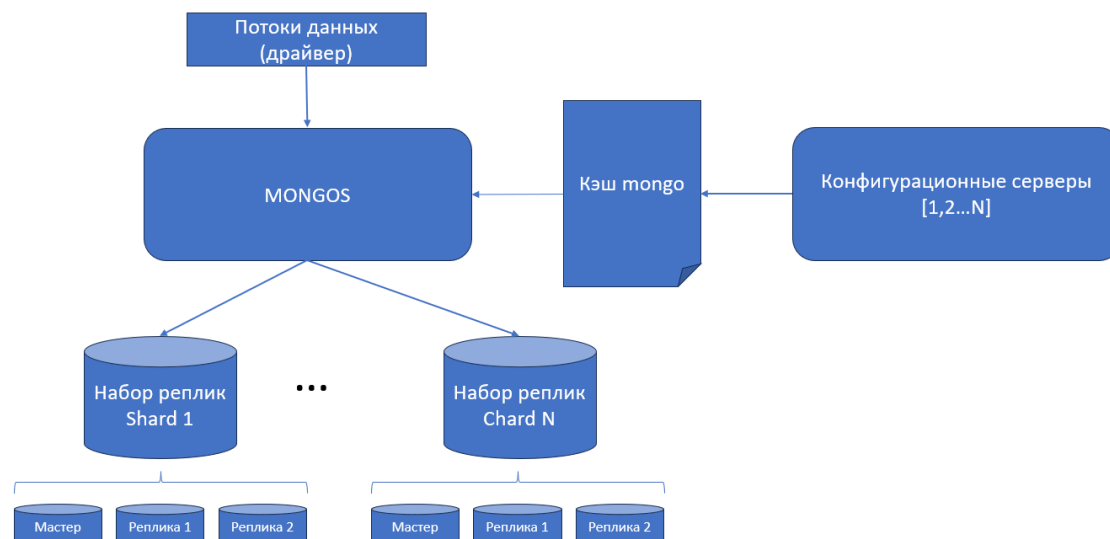


Рисунок 49. Типовая шардированная схема хранилища данных MongoDB с балансировщиком и репликацией

Данная схема использует известный принцип репликации, где в процессе работы системы подписок, набор данных сервера-мастера поддерживается в актуальном состоянии для любого множества серверов-реплик. В приведенной выше схеме, набор реплик шард 1 содержит в себе мастер-сервер с коллекциями и документами шарда, а также сервер-реплику 1 и сервер-реплику 2, на которых сервер-мастер в режиме реального времени обеспечивает актуализацию массивов данных, как копий данных сервера-мастера. Применение данной схемы, помимо прочих традиционных плюсов применения репликации, в случае сбоя сервера-мастера позволяет в тот же самый момент любой готовой реплике временно (до восстановления) занять его место в кластере. Таким образом, доступ к фрагменту данных не будет потерян даже на короткий промежуток времени.

Интересна схема проверок мастер-сервера и реплик на работоспособность. Применяется система проверки сердцебиения (heartbeat). С определенной периодичностью все серверы, находящиеся в системе шарда «набор реплик» сканируют сердцебиение друг друга. В случае отсутствия ответа от сервера-мастера, одна из готовых реплик, в соответствии с определенным заранее приоритетом (или в результате голосования сервера-реплики 1 и сервера-реплики 2), временно занимает место сервера в шарде.

С целью экономии, вместо второго сервера-реплики, в шардированной схеме с репликацией, часто применяется сервер-арбитр. Принцип его работы аналогичен реплике – он постоянно проверяет сердцебиения сервера-мастера и его реплики, и, в случае выхода из строя первого, дает реплике команду занять его место (голосует за временную замену сервера-мастера), но при этом, в

отличие от полноценной реплики, сервер-арбитр не содержит данные.

7.3. Вопросы для самостоятельного изучения по теме

1. Дайте краткое определение «репликации» баз данных? Для чего она нужна? Есть ли смысл в репликации традиционных хранилищ данных?
2. Как осуществляется репликация данных в СУБД PostgreSQL? Какие ключевые инструкции используются?
3. Что такое heartbeat в схеме шардирования с репликацией? Как он технически реализован?

ОТВЕТЫ НА ТЕСТОВЫЕ ЗАДАНИЯ

1 лекция. 1.Б, 2.Б, 3.А, 4.Г, 5.А, 6.Б, 7.А, 8.В, 9.А

2 лекция. 1.В, 2.Г, 3.А, 4.В, 5.Г, 6.Б, 7.А, 8.А

3 лекция. 1.Б, 2.Б, 3.Г, 4.А, 5.Б, 6.В, 7.Г

4 лекция. 1.Б, 2.В, 3.В, 4.В, 5.Б, 6.А, 7.В

5 лекция. 1.Б, 2.В, 3.Д, 4.Д, 5.Б, 6.А

6 лекция. 1.А, 2.Г, 3.В, 4.В, 5.В, 6.А, 7.А, 8.Г

СПИСОК ЛИТЕРАТУРЫ

1. Kroenke D.M., Auer D.J. Database Processing: Fundamentals, Design and Implementation. PEARSON, 2019. - 691 с.: ил..
2. Кренке Д. Теория и практика построения баз данных - Спб.: Питер, 2005. - 859 с.: ил..
3. Ponniah P. Data Warehousing: Fundamentals for IT Professionals. Wiley, 2010. - 570 с.: ил..
4. Kroenke D.M., Auer D.J. Database Concepts. англ. - PEARSON, 2018. - 579 с.: ил..
5. Петкович Д. Microsoft SQL Server 2012. Руководство для начинающих. - Спб.: БХВ-Петербург, 2013. - 817 с.: ил..
6. Сарка Д., Лах М., Йеркич Г. Microsoft SQL Server 2012. Реализация хранилищ данных - Спб.: Наука, 2014. - 805 с.: ил..
7. Харенслак Б., Руйтер Д. Apache Airflow и конвейеры обработки данных. - Лань, 2022. - 505 с.: ил..

СВЕДЕНИЯ ОБ АВТОРЕ



Смирнов Михаил Вячеславович, кандидат экономических наук, доцент кафедры “Предметно-ориентированные информационные системы” Института кибербезопасности и цифровых технологий РТУ-МИРЭА.